BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC PHENIKAA



BÁO CÁO

HỌC PHẦN: ĐỒ ÁN LIÊN NGÀNH

Mã học phần: CSE703014

ĐỀ TÀI:
Khảo sát giải mã suy đoán không sử dụng bộ nhớ trọng số bổ sung trên GPU giới hạn tài nguyên

Sinh viên thực hiện: Nguyễn Quốc Hiếu

Mã số sinh viên: 23010170

Giảng viên hướng dẫn: Nguyễn Công Lượng

HÀ NỘI, 2026

Lời cảm ơn

Em xin chân thành cảm ơn thầy Nguyễn Công Lượng đã hướng dẫn, góp ý và hỗ trợ em trong quá trình thực hiện đồ án. Những nhận xét của thầy giúp em định hướng rõ hơn việc xây dựng hệ thống, thiết kế các phép đo và trình bày kết quả nghiên cứu.

Em cũng xin cảm ơn các thầy cô của Trường Đại học Phenikaa đã trang bị kiến thức nền tảng cho em trong quá trình học tập. Em trân trọng sự hỗ trợ của bạn bè và gia đình trong thời gian thực hiện đồ án.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Em mong nhận được góp ý của thầy cô để tiếp tục hoàn thiện đề tài.

Hà Nội, ngày 18 tháng 10 năm 2026

Nguyễn Quốc Hiếu

Tóm tắt nội dung

Đồ án nghiên cứu cách tăng tốc quá trình sinh văn bản của mô hình ngôn ngữ lớn (LLM) trên GPU laptop có dung lượng bộ nhớ hạn chế. Trong giải mã tự hồi quy thông thường, mô hình dự đoán từng token một, khiến thời gian sinh có thể kéo dài. Giải mã suy đoán tìm cách giảm số bước tuần tự bằng cách tạo trước một số token ứng viên rồi dùng mô hình đích kiểm tra các token đó. Tuy nhiên, cách dùng một mô hình draft riêng làm tăng nhu cầu bộ nhớ. Vì vậy, đồ án tập trung khảo sát những phương án tạo ứng viên không cần nạp thêm trọng số mô hình.

Hệ thống hiện thực và so sánh giải mã vanilla, phương pháp bỏ qua một số lớp của chính mô hình đích (layer-skip), Prompt Lookup Decoding (PLD) và bộ định tuyến kết hợp các hành động giải mã. Trong phương pháp layer-skip, các lớp được chọn của mô hình đích được lược bỏ khi tạo draft; trong PLD, draft được tìm từ các chuỗi token đã xuất hiện trong ngữ cảnh. Hệ thống cũng quản lý KV cache để tái sử dụng phần tiền tố đã được xác minh. Cách tổ chức này loại bỏ nhu cầu lưu thêm trọng số draft, nhưng bản thân quá trình tạo draft, kiểm chứng và lưu trạng thái tạm thời vẫn có chi phí tính toán và bộ nhớ.

Đồ án đánh giá hệ thống trên NVIDIA RTX 4050 Laptop GPU với Qwen2.5-3B-Instruct ở định dạng NF4. Benchmark paired gồm 150 prompt thuộc GSM8K, HumanEval và CNN/DailyMail, tối đa 48 token đầu ra cho mỗi prompt.

Trong artifact benchmark đầy đủ, PLD đạt speedup end-to-end trung bình $1.063\times$ (CI bootstrap 95%: $[1.036, 1.092]$), còn router đạt $1.050\times$ ($[1.023, 1.079]$) so với vanilla. Tỷ số throughput paired router/PLD là 0.989 ($[0.985, 0.993]$), do đó router chưa vượt PLD trong giao thức này. Exact sequence match của PLD và router là 83.3%. Kết quả cho thấy PLD có thể tăng throughput trung bình trong cấu hình đã đo, nhưng không bảo đảm đầu ra trùng vanilla hoặc mọi tập dữ liệu đều nhanh hơn. Kết luận chỉ áp dụng cho mô hình, phần cứng và giao thức benchmark nêu trên.

Vì speedup của PLD không bảo đảm đầu ra trùng vanilla, đồ án nghiên cứu thêm cách làm cho giải mã suy đoán trên mô hình NF4 cho đúng từng token và KV của greedy tuần tự bitsandbytes gốc, không dùng thêm trọng số draft. Vòng giải mã B2 gồm các phép attention, chuẩn hóa và LM head gọi từng hàng với đúng hình dạng của bước một-token (chính các toán tử gây lệch khi kiểm tra theo khối), quay về bước $M{=}1$ gốc khi không có đề xuất, chiếu LM head chỉ tới hàng bị loại đầu tiên, và một kernel Triton xử lý ba hàng tùy chọn chỉ để tăng tốc thêm khoảng 4%. Trên hai cohort held-out GSM8K (mỗi cohort 50 prompt, dùng đúng một lần), B2 khớp token của bitsandbytes ở $50/50$ prompt, nhanh hơn bitsandbytes eager khoảng 1.12–$1.15\times$ và dùng thêm 0.07 MiB bộ nhớ đỉnh. B2 chậm hơn một baseline

`torch.compile` không exact ($0.91\times$ tốc độ của nó trên held-out), và bằng chứng về tính exact chỉ là thực nghiệm hữu hạn trên một GPU; lập luận theo toán tử còn các phần chưa được chứng minh.

Cách làm của B2 chạy từng hàng nên bỏ lợi ích đọc trọng số một lần. Vì thế đồ án đề xuất thêm một bộ kernel 4-bit *bất biến theo số hàng* M (LM head, attention, RMSNorm và GEMM NF4 viết bằng CUDA C++): kết quả mỗi hàng không phụ thuộc M nhờ khối hàng đệm cố định, thứ tự rút gọn cố định, không split-K, không atomics, được kiểm từng bit trên hàng chục nghìn hàng mỗi kernel. Trên bộ kernel này, vòng PLD exact chạy trong CUDA graph cho đúng token và KV từng bit của giải mã tuần tự $M{=}1$ cùng kernel trên 1200 cặp prompt-mô hình (tám mô hình Qwen2.5-0.5B/1.5B/3B, OLMo-2-1B, Phi-3.5-mini, Llama-3.2-1B/3B và Mistral-7B; GSM8K, HumanEval, CNN/DailyMail), nhanh hơn baseline exact 1.14–$1.32\times$ và nhanh hơn vòng exact chạy từng hàng 1.26–$1.39\times$. Tham chiếu exact ở đây là bộ kernel mới, không phải bitsandbytes gốc: hai tham chiếu cho cùng chuỗi ở chỉ 30–44% số prompt GSM8K, và không phát hiện giảm độ chính xác tác vụ ở cỡ mẫu nhỏ. Một mô hình nháp thần kinh vẫn exact nhưng chậm hơn ($0.66\times$) trên đích 3B này. Tất cả kết quả trên một GPU RTX 4050 (6 GB), mô hình tới 7B.

Từ khóa: giải mã suy đoán, mô hình ngôn ngữ lớn, KV cache, GPU biên, Prompt Lookup Decoding, lượng tử hóa NF4, giải mã exact, kernel bất biến theo số hàng.

Danh mục ký hiệu và viết tắt

Từ viết tắt

Ký hiệu	Diễn giải	Ký hiệu	Diễn giải
LLM	Mô hình ngôn ngữ lớn.	PLD	Prompt Lookup Decoding.
KV	Key–Value của attention.	VRAM	Bộ nhớ đồ họa GPU.
TPS	Token sinh mỗi giây.	EOS	Token kết thúc chuỗi.
BNB	Backend bitsandbytes.	HQQ	Phương pháp lượng tử hóa.
NF4	Định dạng lượng tử hóa 4 bit.	SDPA	Scaled Dot-Product Attention.
CKA	Centered Kernel Alignment.	RQ	Câu hỏi nghiên cứu.
B1	Tham chiếu greedy với lịch hình dạng cố định (Chương 6).	B2	Tham chiếu greedy bitsandbytes gốc và vòng giải mã exact bám theo nó (Chương 6).
RE	Tính bằng nhau theo hàng (row-equivalence).	CI	Khoảng tin cậy (bootstrap).
PTX	Mã trung gian của GPU NVIDIA.	GEMV	Nhân ma trận–vector.
SHA256	Hàm băm dùng để so KV và ghim tệp.	GSM8K	Tập bài toán số học.

Ký hiệu

Ký hiệu	Diễn giải	Ký hiệu	Diễn giải
x	Prompt đầu vào.	y	Token đã append vào output.
p	Token target đang pending.	P	Prefix đã xử lý trong KV.
d_i	Token draft thứ i.	A	Prefix draft được chấp nhận.
$p'(c)$	Correction/bonus pending.	C_{tgt}	KV cache chuẩn target.
B	Ngân sách token đầu ra.	r	Số vị trí ngân sách còn lại.
K	Horizon tối đa của draft.	N_{emit}	Token append trong một cycle.

T_{cycle}	Thời gian wall-clock một cycle.	T_v	Latency vanilla mỗi token.
S	Speedup throughput ước lượng.	M	Số hàng của một khối kiểm tra ($M = K + 1$).
L_t	Độ dài logic của KV cache tại bước t.		

Mục lục

Lời cảm ơn i

Tóm tắt nội dung ii

Danh mục ký hiệu và viết tắt iv

1 Giới thiệu 1

1.1 Bối cảnh và động lực nghiên cứu . . . 1
1.2 Mục tiêu và câu hỏi nghiên cứu . . . 1
1.3 Phạm vi và giới hạn . . . 2
1.4 Cấu trúc báo cáo . . . 2

2 Cơ sở lý thuyết và các nghiên cứu liên quan 3

2.1 Cơ sở lý thuyết . . . 3
2.1.1 Giải mã tự hồi quy và speculative decoding . . . 3
2.1.2 Self-speculative decoding và bỏ qua lớp . . . 4
2.1.3 Prompt Lookup Decoding . . . 4
2.1.4 KV cache và numerical correctness . . . 5
2.2 Các nghiên cứu liên quan . . . 5
2.2.1 Draft-based và self-speculative decoding . . . 5
2.2.2 Retrieval-based và alternative drafting . . . 5
2.2.3 Systems efficiency và memory management . . . 6
2.2.4 So sánh các hướng nghiên cứu . . . 6
2.3 Khoảng trống và định hướng nghiên cứu . . . 7
2.3.1 Feasibility phụ thuộc backend . . . 7
2.3.2 Dự đoán chi phí trên workload chưa quan sát . . . 8
2.3.3 Numerical fidelity dưới lượng tử hóa và verification theo lô . . . 8
2.3.4 Kernel bất biến theo batch và theo số hàng . . . 8
2.3.5 Định hướng và đóng góp trong phạm vi đồ án . . . 8

3 Phân tích và thiết kế hệ thống 10

3.1 Yêu cầu và ràng buộc thiết kế 10
3.1.1 Yêu cầu chức năng 10
3.1.2 Yêu cầu phi chức năng 11
3.1.3 Ràng buộc thiết kế 11
3.2 Kiến trúc tổng thể 12
3.2.1 Các thành phần chính 12
3.2.2 Luồng điều khiển và dữ liệu 12
3.3 Quy trình giải mã suy đoán 13
3.4 Chia sẻ KV cache và các bất biến 15
3.5 Mô hình chi phí và điều kiện hòa vốn 17
3.6 Thiết kế bộ định tuyến 18

4 Hiện thực chương trình **20**
4.1 Môi trường và tổ chức mã nguồn 20
4.2 Hiện thực các phương pháp giải mã 22
4.2.1 Vanilla Decoding 22
4.2.2 Layer-Skip Self-Speculative Decoding 22
4.2.3 Prompt Lookup Decoding 23
4.2.4 Target Verification 23
4.3 Hiện thực KV cache và chọn lớp 24
4.4 Router và thu thập dữ liệu 25
4.5 Tích hợp inference backend 27
4.6 Giao diện chạy và kiểm tra 28

5 Phương pháp thực nghiệm và kết quả **31**
5.1 Thiết lập thực nghiệm 31
5.1.1 Cấu hình benchmark chính 31
5.2 Tập dữ liệu và giao thức đánh giá 32
5.3 Phương pháp đối chứng và chỉ số 32
5.4 Kết quả benchmark chính 33
5.4.1 So sánh paired trên ba tập dữ liệu 33
5.4.2 Layer-skip và router trên cohort riêng 36
5.5 Phân tích chi phí và điều kiện hòa vốn 36
5.6 Đánh giá bộ nhớ và tính đúng đắn 38
5.6.1 Bộ nhớ 38
5.6.2 Numerical fidelity và sequence identity 38
5.6.3 Kiểm tra logic KV cache 39

5.7 Thảo luận và giới hạn . . . 39

6 Giải mã suy đoán exact trên mô hình NF4 không dùng thêm bộ nhớ GPU 41
6.1 Động lực và mục tiêu . . . 41
6.2 Bài toán và định nghĩa . . . 42
6.3 Phương pháp . . . 42
6.4 Lập luận về tính đúng đắn . . . 43
6.4.1 Thứ tự rút gọn của kernel bitsandbytes . . . 44
6.5 Thiết lập thực nghiệm . . . 44
6.6 Kết quả . . . 45
6.6.1 Tính đúng đắn . . . 45
6.6.2 Toán tử nào làm vỡ tính exact . . . 45
6.6.3 Tốc độ và so sánh với B1 . . . 46
6.6.4 So sánh với baseline biên dịch . . . 46
6.6.5 Ngữ cảnh dài và các mô hình khác . . . 47
6.6.6 Bộ nhớ . . . 47
6.6.7 Phân tích chi phí . . . 47
6.6.8 Thăm dò: chụp CUDA graph (ước lượng, chưa cài đặt) . . . 48
6.7 Các kết quả bất lợi và hướng không hiệu quả . . . 48
6.8 Giới hạn và mối đe dọa tính hợp lệ . . . 49
6.9 Tóm tắt chương . . . 49

7 Kernel 4-bit bất biến theo số hàng cho giải mã suy đoán exact 51
7.1 Vấn đề: kernel đổi chiến lược theo M . . . 51
7.2 Định nghĩa exact và tham chiếu . . . 52
7.3 Thiết kế kernel bất biến . . . 53
7.4 Chi phí một bước giải mã . . . 55
7.5 Vòng giải mã exact trên CUDA graph . . . 55
7.6 Kết quả đầu-cuối . . . 56
7.7 Độ phụ thuộc vào phiên bản bitsandbytes . . . 58
7.8 Kết quả âm . . . 58
7.9 Giới hạn và mối đe dọa tính hợp lệ . . . 58
7.10 Tóm tắt chương . . . 59

8 Kết luận và hướng phát triển 60
8.1 Kết luận . . . 60
8.2 Hạn chế . . . 61
8.3 Hướng phát triển . . . 62

Tài liệu tham khảo **63**

Danh sách hình vẽ

1 Vanilla đi thẳng tới target; draft speculative phải qua bước xác minh. 12
2 Trạng thái KV trước và sau verification: target xử lý pending input p, chấp nhận d_1, d_2, từ chối d_3 và chọn correction c. Sau commit/rollback, active KV chứa $P \| p \| d_1 \| d_2$; c được xử lý ở forward target kế tiếp. 16
3 Các thành phần thời gian khái niệm trong một speculative cycle. 18

4 Cấu trúc package thực tế dưới `src/zassd`. 21
5 Các gate router theo thứ tự thực thi. Nhánh PLD có thể chọn draft trước gate so sánh với vanilla. 26

6 Speedup end-to-end của các phương pháp theo từng benchmark trong cohort 150 prompt. 34
7 Phân bố speedup end-to-end paired theo prompt; đường đứt đoạn biểu thị hòa vốn $1.0\times$. 35
8 Speedup theo tập trên vùng tập trung 0.85–1.20$\times$; trục tung được thu hẹp để thấy rõ chênh lệch quanh đường hòa vốn $1.0\times$. Ký hiệu đơn sắc: hình tròn PLD, hình vuông PLD gate, tam giác router; thanh ngang là CI bootstrap 95%. 35

9 Tốc độ so với BNB eager trên held-out GSM8K 400–449 (50 prompt; điểm là trung bình nhân, thanh là khoảng tin cậy 95%). Dòng trên cùng ghi số prompt có token khớp BNB eager. BNB biên dịch nhanh hơn nhưng chỉ khớp $25/50$. 47
10 Tốc độ so với BNB eager theo độ dài prompt (3 prompt dev mỗi độ dài, tóm tắt nhiều bài CNN/DM ghép lại). 48

11 Phân bố tỷ lệ chuỗi lệch quỹ đạo greedy tuần tự của PLD ngây thơ trên 15 ô (mô hình $\times$ tập dữ liệu). 52
12 Nguyên tắc thiết kế: khối M hàng được đệm 0 tới kích thước cố định M_P và đi qua cùng một dãy lệnh; kết quả mỗi hàng không phụ thuộc M. 53
13 Thời gian một bước giải mã trong CUDA graph, Qwen2.5-3B NF4, theo số hàng của khối. 55

14 Phân bố tăng tốc theo prompt của vòng PLD exact so với baseline $M{=}1$ cùng kernel. 57

Danh sách bảng

2.1 So sánh các hướng nghiên cứu liên quan 7

3.1 Yêu cầu chức năng của hệ thống 10
3.2 Yêu cầu phi chức năng 11
3.3 Ràng buộc thiết kế và hệ quả 11
3.4 Các thành phần chính của kiến trúc 12
3.5 Các bất biến của KV cache 15
3.6 Phân biệt ba mức đúng đắn cần đánh giá 16

4.1 Môi trường triển khai và công nghệ sử dụng 21
4.2 Mapping giữa package mã nguồn và chức năng triển khai 25
4.3 Tín hiệu và vai trò trong router hiện thực 26
4.4 Khác biệt triển khai giữa hai đường backend được khảo sát 28
4.5 Trích kết quả paired benchmark thực tế: vanilla và PLD trên cùng prompt . 29
4.6 Trạng thái kiểm thử được xác minh trong workspace hiện tại 29

5.1 Cấu hình của benchmark paired chính 31
5.2 Các cohort thực nghiệm và phạm vi sử dụng 32
5.3 Speedup end-to-end và exact sequence match trong benchmark chính 34
5.4 Thống kê cycle gộp suy ra từ raw benchmark paired 37
5.5 Một số số đo Nsight chẩn đoán trên một prompt cố định 37
5.6 Kết quả numerical fidelity trên cohort 30 prompt; thời gian có tracing chỉ mang tính chẩn đoán 39
5.7 Tổng hợp kết quả theo câu hỏi nghiên cứu 40

6.1 Trạng thái lập luận theo toán tử. **[S]**: theo cấu trúc mã; **[E]**: chỉ có bằng chứng thực nghiệm; **[O]**: còn mở. 44
6.2 Bằng chứng tính đúng đắn của B2 (so với greedy bitsandbytes gốc). 45
6.3 So sánh từng bit giữa toán tử gọi ba hàng và gọi từng hàng trên cùng đầu vào (đường chuẩn, 960 hàng). 46

7.1 Độ lệch từng bit so với chạy từng hàng, bitsandbytes 0.50.0 NF4, GSM8K, 960 hàng mỗi ô. “Proj”: tỷ lệ phần tử sai của các phép chiếu NF4; RMSNorm là số phần tử sai. 52
7.2 Tham chiếu bất biến so với greedy bitsandbytes gốc, Qwen2.5-3B NF4. . . 53
7.3 Kiểm chứng bất biến theo M (số hàng lệch / tổng số hàng kiểm). 54
7.4 Vòng PLD exact so với baseline $M{=}1$ cùng kernel. Tăng tốc theo prompt là trung vị [min, max]. Cận trên Wilson 95% của tỷ lệ lệch: 1.9% ($n{=}200$), 3.7% ($n{=}100$). 56
7.5 Tỷ lệ phần tử NF4 khác từng bit giữa khối M hàng và chạy từng hàng (stock bitsandbytes, hình dạng $11008{\times}2048$, dữ liệu ngẫu nhiên). 58

Giới thiệu

1.1 Bối cảnh và động lực nghiên cứu

Mô hình ngôn ngữ lớn thường sinh đầu ra theo cơ chế tự hồi quy, trong đó mỗi token mới phụ thuộc vào các token đã sinh trước đó. Sự phụ thuộc tuần tự này làm tăng độ trễ khi sinh chuỗi dài. Speculative decoding giảm số bước tuần tự bằng cách tạo trước các token ứng viên rồi dùng mô hình đích kiểm tra chúng. Cách triển khai dùng một mô hình draft riêng có thể làm tăng đáng kể nhu cầu bộ nhớ vì phải lưu đồng thời trọng số của draft và target.

Self-speculative decoding tạo draft từ chính mô hình đích, chẳng hạn bằng cách bỏ qua một số lớp; Prompt Lookup Decoding (PLD) tạo draft bằng cách tra cứu các chuỗi token đã xuất hiện trong ngữ cảnh. Hai cách tiếp cận tránh nạp thêm trọng số draft nhưng có các chi phí khác nhau: layer-skip vẫn thực thi một phần mô hình, còn PLD phụ thuộc vào mức độ lặp lại của workload. Trong cả hai trường hợp, target vẫn cần xác minh các token dự thảo.

Mâu thuẫn kỹ thuật trung tâm là giảm bộ nhớ trọng số không đồng nghĩa với giảm độ trễ. Hiệu quả phụ thuộc vào chi phí draft, verify, backend lượng tử hóa, quản lý KV cache và độ ổn định số học. Vì thế cần một đánh giá có đối chứng để xác định trong điều kiện nào các phương pháp không dùng trọng số draft bổ sung thực sự có lợi.

1.2 Mục tiêu và câu hỏi nghiên cứu

Mục tiêu của đồ án là xây dựng và đánh giá một hệ thống giải mã suy đoán cho GPU có VRAM hạn chế, tập trung vào hiệu năng, chi phí bộ nhớ và độ khớp chuỗi đầu ra. Các câu hỏi nghiên cứu được đặt ra như sau:

RQ1. Khi nào layer-skip và PLD tạo speedup so với vanilla decoding trên GPU giới hạn VRAM?

RQ2. Chi phí draft, verify và overhead quyết định ranh giới khả thi (feasibility boundary) của từng hành động như thế nào?

RQ3. Backend lượng tử hóa, chế độ thực thi theo lô/tuần tự và trạng thái KV cache ảnh hưởng thế nào đến numerical fidelity và exact token match?

RQ4. Router có cải thiện so với phương pháp cố định tốt nhất trên cùng workload không?

RQ5. Có thể xây dựng một vòng giải mã suy đoán trên mô hình NF4 cho đúng từng token và KV của greedy tuần tự gốc, không thêm bộ nhớ đáng kể, và với tốc độ như thế nào?

RQ6. Có thể thiết kế kernel 4-bit mà kết quả mỗi hàng không phụ thuộc số hàng trong khối, để kiểm tra theo khối vừa exact vừa đọc trọng số một lần, và mức tăng tốc đạt được là bao nhiêu?

1.3 Phạm vi và giới hạn

Đồ án tập trung vào suy luận batch nhỏ trên GPU laptop có VRAM hạn chế, không huấn luyện lại mô hình ngôn ngữ và không nạp auxiliary draft-model weights trong các phương pháp mục tiêu. Các phương thức được khảo sát gồm vanilla, layer-skip, PLD và router kết hợp. Đánh giá bao gồm throughput/latency, bộ nhớ và sequence fidelity dưới greedy decoding. Hai backend lượng tử hóa BNB NF4 và HQQ+GemLite W4A16 được khảo sát trong các thí nghiệm riêng. Benchmark paired chính dùng Qwen2.5-3B-Instruct NF4 trên RTX 4050; các thí nghiệm backend/numerical fidelity có thể khác về giao thức, cỡ mẫu hoặc giới hạn token và được báo cáo tách biệt.

Phần giải mã exact ở Chương 6 và Chương 7 chỉ xét backend bitsandbytes NF4, họ mô hình Qwen2.5 (Chương 7 thêm OLMo-2), batch 1 và greedy decoding trên một GPU, và so sánh tốc độ với chính greedy bitsandbytes gốc. Kết luận chỉ áp dụng cho cấu hình, model revision, backend, prompt set và giao thức được nêu trong từng thí nghiệm. Đồ án không khẳng định lossless behavior trên mọi backend, precision hoặc execution mode; cũng không khái quát speedup đo được trên RTX 4050 thành bảo đảm cho thiết bị khác.

1.4 Cấu trúc báo cáo

Báo cáo gồm tám chương. Chương 1 nêu động lực, câu hỏi nghiên cứu và phạm vi. Chương 2 trình bày cơ sở lý thuyết, công trình liên quan và các vấn đề nghiên cứu được khảo sát. Chương 3 mô tả yêu cầu, kiến trúc, thuật toán giải mã, quản lý KV cache, mô hình chi phí và router. Chương 4 trình bày hiện thực chương trình. Chương 5 mô tả giao thức và kết quả thực nghiệm. Chương 6 trình bày vòng giải mã suy đoán exact trên NF4 (B2) và bằng chứng đánh giá nó, trả lời RQ5. Chương 7 trình bày bộ kernel 4-bit bất biến theo số hàng và vòng giải mã exact trên CUDA graph, trả lời RQ6. Chương 8 tổng kết, nêu giới hạn và hướng phát triển.

Cơ sở lý thuyết và các nghiên cứu liên quan

2.1 Cơ sở lý thuyết

2.1.1 Giải mã tự hồi quy và speculative decoding

Giải mã tự hồi quy

Với chuỗi token $x_{1:T}$, mô hình ngôn ngữ tự hồi quy phân rã xác suất như sau:

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{1:t-1}). \tag{2.1}$$

Ở mỗi bước, mô hình tính phân phối của token kế tiếp rồi chọn token theo greedy decoding hoặc sampling. Token mới được thêm vào lịch sử và trở thành điều kiện cho bước tiếp theo. Do đó, các bước sinh nối tiếp có phụ thuộc tuần tự, dù phép tính trong từng lượt forward có thể song song. KV cache tránh tính lại key/value của tiền tố, nhưng không loại bỏ phụ thuộc giữa các token mới. Dự đoán theo khối là một hướng giảm số lượt tuần tự [1].

Draft–verify và tính đúng đắn

Speculative decoding dùng mô hình draft có phân phối q để đề xuất tối đa K token, sau đó mô hình target có phân phối p kiểm tra chúng theo thứ tự. Trong speculative sampling, token draft y tại ngữ cảnh h được chấp nhận với xác suất:

$$a(y \mid h) = \min\left(1, \frac{p(y \mid h)}{q(y \mid h)}\right). \tag{2.2}$$

Nếu bị từ chối, token hiệu chỉnh được lấy từ phân phối phần dư:

$$r(y \mid h) = \frac{\max(p(y \mid h) - q(y \mid h), 0)}{\sum_v \max(p(v \mid h) - q(v \mid h), 0)}. \tag{2.3}$$

Quy tắc hiệu chỉnh này bảo toàn phân phối target theo các giả định của thuật toán [2, 3]. Với greedy decoding, kiểm tra nhằm giữ lựa chọn argmax của target tại từng vị trí; sequence identity là tiêu chí mạnh hơn việc chỉ dùng cùng thuật toán trên danh nghĩa. Vì khác biệt floating-point/backend có thể đổi argmax khi logits gần nhau, tính đúng đắn cần được báo cáo theo cấu hình thực thi cụ thể.

Chi phí của một chu kỳ

Nếu chu kỳ phát trung bình $\mathbb{E}[N_{\text{emit}}]$ token và mất thời gian $\mathbb{E}[T_{\text{cycle}}]$, tốc độ suy đoán phụ thuộc vào tỷ số hai đại lượng này, không chỉ vào acceptance. Draft, verify và overhead cache/điều phối đều góp phần vào T_{cycle}. Mô hình chi phí và điều kiện hòa vốn được xây dựng cụ thể ở Mục 3.5.

2.1.2 Self-speculative decoding và bỏ qua lớp

Self-speculative decoding tạo draft từ một nhánh rút gọn của chính target, nhờ đó không cần nạp một bộ trọng số draft riêng. Với tập lớp bỏ qua S, có thể mô tả phép biến đổi draft ở lớp l bằng:

$$h_{\text{draft}}^{(l+1)} = \begin{cases} h_{\text{draft}}^{(l)}, & l \in S, \\ F_l(h_{\text{draft}}^{(l)}), & l \notin S, \end{cases} \tag{2.4}$$

trong đó F_l là khối Transformer. Bỏ nhiều lớp có thể giảm chi phí draft nhưng cũng làm dự đoán lệch target và giảm số token được chấp nhận; bỏ ít lớp có thể giữ draft gần target hơn nhưng tiết kiệm ít tính toán.

Cần phân biệt draft tạo bằng cách thực thi một phần target với early exit được hỗ trợ bằng huấn luyện bổ sung. Đồ án khảo sát layer-skip trong cấu hình hiện thực, không giả định chất lượng của draft hoặc lợi ích tốc độ chỉ từ cách chọn lớp. Các công trình và khác biệt về thiết kế được đối chiếu tại Mục 2.2.1.

2.1.3 Prompt Lookup Decoding

Prompt Lookup Decoding (PLD) tìm một n-gram trùng với hậu tố của lịch sử hiện tại trong vùng văn bản nguồn, rồi lấy các token đứng ngay sau lần khớp làm draft [4]. Gọi lịch sử là $H = (x_1, \ldots, x_t)$, nguồn tra cứu là $C = (c_1, \ldots, c_m)$ và n-gram truy vấn độ dài n là $q = (x_{t-n+1}, \ldots, x_t)$. Một vị trí i là match nếu:

$$(c_i, c_{i+1}, \ldots, c_{i+n-1}) = q. \tag{2.5}$$

Các token sau vị trí khớp được đề xuất tối đa đến horizon K và vẫn phải được target xác minh. N-gram ngắn dễ match nhưng có thể ít đặc trưng; n-gram dài đặc trưng hơn nhưng dễ miss. Khi không có continuation phù hợp, hệ thống fallback về vanilla. PLD có chi phí draft thấp nhưng hiệu quả phụ thuộc mức lặp của workload; các hướng retrieval-based khác cũng khai thác nội dung có sẵn để tạo ứng viên [5].

2.1.4 KV cache và numerical correctness

Ở mỗi lớp attention, KV cache lưu key/value của token history để tránh tính lại biểu diễn tiền tố. Cache state phải tương ứng với đúng token history: logical length, các vị trí hợp lệ, causal mask và vị trí RoPE cần nhất quán. Nếu token bị loại sau verify nhưng KV của token đó vẫn được xem là active, các forward sau sẽ xử lý một trạng thái không còn tương ứng với chuỗi đã xác nhận.

Cần phân biệt tính đúng đắn thuật toán với numerical equivalence của một backend cụ thể. Quy tắc speculative sampling có thể bảo toàn phân phối trong giả định toán học; nhưng lượng tử hóa, kernel attention và cách xử lý batched/sequential có thể làm logits hoặc KV sai khác floating-point. Khi logits top-1/top-2 gần nhau, sai khác nhỏ có thể gây đổi token greedy. Vì vậy cần đo cả sequence match và trạng thái cache, đồng thời ghi rõ model, precision, backend và execution mode [6]. Cách ZASSD giữ cache chuẩn, cache draft tạm và rollback được trình bày tại Mục 3.4.

2.2 Các nghiên cứu liên quan

2.2.1 Draft-based và self-speculative decoding

Speculative decoding với draft riêng đặt nền tảng cho quy trình đề xuất rồi xác minh token [2, 3]. Trong self-speculation, LayerSkip dùng huấn luyện để hỗ trợ early exit; Draft and Verify khảo sát bỏ lớp theo hướng training-free; Kangaroo kết hợp hai điểm thoát sớm; ShortGPT đánh giá mức dư thừa của lớp; còn CLaSp chọn lớp theo ngữ cảnh [7, 8, 9, 10, 11]. Các hướng này khác nhau ở cách tạo draft, yêu cầu huấn luyện và chính sách chọn lớp. Vì vậy, chúng là các thiết kế liên quan chứ không phải cấu hình tương đương trực tiếp với layer-skip được đánh giá trong đồ án.

2.2.2 Retrieval-based và alternative drafting

PLD và REST khai thác n-gram hoặc nội dung đã truy xuất để tạo ứng viên [4, 5]. Medusa dùng nhiều đầu dự đoán, còn EAGLE xây dựng draft dựa trên đặc trưng của mô hình [12, 13].

Các phương pháp này mở rộng không gian thiết kế draft nhưng khác nhau về thành phần bổ sung, dữ liệu và huấn luyện cần thiết. Với mục tiêu tránh nạp trọng số draft, PLD là baseline gần nhất; Medusa và EAGLE chủ yếu là đối chiếu về cách tạo ứng viên.

2.2.3 Systems efficiency và memory management

PagedAttention tập trung vào quản lý KV cache khi phục vụ LLM [6]. CKA được dùng để so sánh biểu diễn giữa các lớp/mạng, nhưng độ tương đồng biểu diễn không tự động đảm bảo acceptance hoặc tốc độ cao hơn [14]. Roofline giúp phân tích giới hạn do năng lực tính toán và băng thông bộ nhớ [15]. Các công cụ này cung cấp góc nhìn hệ thống cho việc chọn lớp và diễn giải chi phí, song quyết định cuối cần được xác nhận bằng phép đo toàn chu kỳ.

LLM-42 [16] giải quyết bài toán non-determinism trong LLM serving bằng cơ chế decode–verify–rollback: token được sinh trên fast path không xác định, sau đó verifier replay dưới lịch thực thi fixed-shape để commit hoặc loại bỏ token không nhất quán. Công trình này tập trung vào FP16 models (Llama-3.1-8B) trên hệ thống đa GPU (4x H100), với mục tiêu chính là determinism dưới dynamic batching trong môi trường serving. Bài toán của đồ án khác ở chỗ nghiên cứu tính chính xác số học khi xác minh token trong môi trường lượng tử hóa 4-bit (NF4) trên GPU đơn, nơi bản thân kernel dequantization và reduction order là nguồn sai lệch cần kiểm soát; LLM-42 không đề cập đến NF4 hay bitsandbytes.

2.2.4 So sánh các hướng nghiên cứu

Bảng 2.1 tóm tắt nguồn draft, yêu cầu thành phần/huấn luyện và mối liên hệ với đồ án. Các mô tả phản ánh thiết kế của từng công trình, không phải so sánh hiệu năng trực tiếp.

Bảng 2.1: So sánh các hướng nghiên cứu liên quan

Công trình	**Nguồn draft và yêu cầu bổ sung**	**Liên hệ với đồ án**
Leviathan et al. [2]	Mô hình draft riêng; không bắt buộc joint training.	Nền tảng draft–verify và hiệu chỉnh phân phối.
LayerSkip [7]	Early exit từ target; có huấn luyện hỗ trợ.	Self-speculation cần chuẩn bị mô hình.
Draft and Verify [8]	Draft từ một phần lớp target; hướng training-free.	Gần với layer-skip không thêm trọng số.
Prompt Lookup [4]	N-gram trong prompt/ngữ cảnh; không thêm trọng số hay huấn luyện.	Baseline draft chi phí thấp.
REST [5]	Draft từ datastore; yêu cầu phụ thuộc thiết kế.	Đại diện cho hướng truy xuất nội dung.
Medusa / EAGLE [12, 13]	Prediction heads hoặc feature draft; có thành phần/huấn luyện bổ sung.	Đối chiếu các hướng tạo draft ngoài PLD và layer-skip.
LLM-42 [16]	Decode–verify–rollback với fixed-shape schedule; không thêm trọng số draft.	Công trình gần nhất về verify–rollback; khác đồ án ở mô hình FP16, hệ đa GPU, và không xử lý lượng tử hóa 4-bit.

2.3 Khoảng trống và định hướng nghiên cứu

Đồ án không khẳng định rằng các vấn đề dưới đây chưa từng được giải quyết trong toàn bộ literature. Đây là ba câu hỏi thực nghiệm được chọn trong phạm vi hệ thống và cấu hình nghiên cứu.

2.3.1 Feasibility phụ thuộc backend

Chi phí một hành động speculative không chỉ do thuật toán quyết định. Lượng tử hóa, kernel và execution mode có thể thay đổi latency draft/verify, dung lượng cache và sai khác số học. Vì thế, cùng một cấu hình layer-skip hoặc K có thể khả thi ở backend này nhưng không có lợi ở backend khác. Đồ án cần phân biệt BNB NF4 và HQQ+GemLite ở những thí nghiệm đã chạy, ghi metadata và không gộp các phép đo khác giao thức.

2.3.2 Dự đoán chi phí trên workload chưa quan sát

Acceptance cao không đủ để chọn hành động tối ưu nếu draft hoặc verify đắt. Router cần ước lượng số token phát ra và thời gian chu kỳ cho vanilla, PLD và layer-skip, rồi chỉ chọn hành động có throughput dự kiến cao hơn baseline. Câu hỏi còn lại là mức độ hiệu chuẩn và khả năng tổng quát của ước lượng trên prompt chưa dùng để xây dựng profile; một comparator offline nhìn thấy kết quả toàn prompt chỉ có vai trò chẩn đoán, không phải router triển khai.

2.3.3 Numerical fidelity dưới lượng tử hóa và verification theo lô

Ngay cả khi logic commit/rollback được thiết kế đúng, cache state và phép tính batched có thể tạo khác biệt logits so với vanilla tuần tự. Cần tách lỗi trạng thái cache khỏi sai khác do precision/backend, kiểm tra vị trí divergence và đánh giá sequence match theo cấu hình đầy đủ. Đồ án xem đây là vấn đề cần đo và mô tả minh bạch, không suy ra lossless behavior chỉ từ chứng minh của speculative sampling trong số học lý tưởng.

2.3.4 Kernel bất biến theo batch và theo số hàng

Nguyên lý rằng kết quả của một token không được phụ thuộc vào số token khác được xử lý cùng lúc đã được nêu trong công trình về suy luận xác định: kernel RMSNorm, matmul và attention thường đổi thứ tự rút gọn theo kích thước batch, và các kernel bất biến theo batch cố định thứ tự đó [17]; Yuan và cộng sự đo độ lệch độ chính xác giữa các cấu hình GPU, kích thước batch và độ chính xác số học [18]. LLM-42 [16] xác minh trên cửa sổ kích thước cố định và không tích hợp với giải mã suy đoán dùng mô hình nháp. Với mô hình lượng tử hóa trọng số, quan sát rằng kiểm tra theo khối làm đổi logit và lật argmax đã được ghi nhận định tính trên backend Metal [19]. Khoảng trống mà Chương 7 nhắm tới là: (i) đo theo toán tử trên CUDA với trọng số NF4, (ii) bộ kernel 4-bit bất biến theo số hàng *trong một chuỗi* (không chỉ theo batch), và (iii) vòng giải mã suy đoán exact dùng bộ kernel đó. Phần này dựa trên việc đọc nội dung chọn lọc các công trình trên, không phải khảo sát đầy đủ.

2.3.5 Định hướng và đóng góp trong phạm vi đồ án

Từ các câu hỏi trên, đồ án tập trung vào tích hợp vanilla, layer-skip, PLD và router; quản lý KV cache chuẩn/tạm; xây dựng phép so sánh paired; và phân tích chi phí cùng numerical fidelity theo model/backend đã khảo sát. Đóng góp được xác định ở mức hiện thực và đánh giá hệ thống. Kết quả benchmark cụ thể được trình bày ở Chương 5, còn vòng giải mã exact trên NF4 và đánh giá của nó ở Chương 6, bộ kernel bất biến theo số hàng ở Chương 7, không

dùng làm bằng chứng rằng router hoặc một phương pháp draft luôn tốt hơn trên workload khác.

Phân tích và thiết kế hệ thống

3.1 Yêu cầu và ràng buộc thiết kế

Bối cảnh thiết kế

Hệ thống hướng tới suy luận batch nhỏ trên GPU laptop có VRAM hạn chế. Cấu hình benchmark chính sử dụng NVIDIA GeForce RTX 4050 Laptop GPU và Qwen2.5-3B-Instruct NF4. Các phương pháp cần so sánh gồm vanilla, layer-skip, Prompt Lookup Decoding (PLD) và router. Mục tiêu là tránh nạp thêm trọng số draft, đồng thời đo riêng tốc độ, bộ nhớ và độ khớp đầu ra. Thiết kế không giả định speculative decoding luôn nhanh hơn vanilla; trong NF4, tính đúng đắn phải được xác nhận theo cấu hình đo vì kiểm chứng theo lô có thể gây sai khác số học.

3.1.1 Yêu cầu chức năng

Bảng 3.1 tóm tắt các thao tác hệ thống cần hỗ trợ.

Bảng 3.1: Yêu cầu chức năng của hệ thống

Mã	Thành phần	Yêu cầu
FR1	Model/backend	Nạp model và chọn quantization/backend.
FR2	Vanilla	Sinh tuần tự làm baseline.
FR3	Layer-skip	Tạo draft bằng tập lớp rút gọn của target.
FR4	PLD	Draft từ n-gram; fallback khi lookup miss.
FR5	Verification	Xử lý acceptance, correction/bonus, EOS và token budget.
FR6	KV cache	Quản lý cache chuẩn, commit/rollback và vị trí hợp lệ.
FR7	Router	Chọn phương thức giải mã và horizon.
FR8	Metrics	Ghi latency, TPS, VRAM và độ khớp đầu ra.

3.1.2 Yêu cầu phi chức năng

Bảng 3.2: Yêu cầu phi chức năng

Yêu cầu	Tiêu chí thiết kế
Tính đúng đắn	Đối chiếu token IDs và kiểm tra cache length/position sau verify, rollback.
Bộ nhớ	Không nạp draft weights; báo cáo peak VRAM và các thành phần bộ nhớ chính.
Hiệu năng	Đo wall-clock toàn cycle; dùng acceptance để giải thích kết quả.
Tái lập	Lưu commit, model/backend, cấu hình sinh, prompt IDs/hash và warmup.
Chẩn đoán	Ghi vị trí divergence cùng logits/KV trace khi cần.
Mô-đun	Tách model, draft, verifier, cache, router và profiler.

3.1.3 Ràng buộc thiết kế

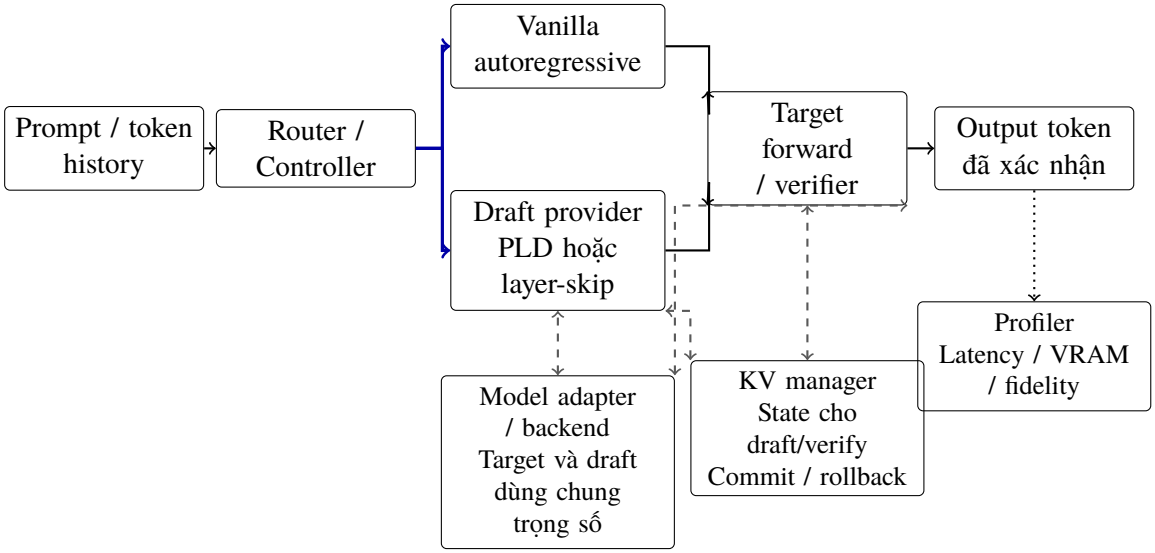
Bảng 3.3: Ràng buộc thiết kế và hệ quả

Mã	Ràng buộc	Hệ quả đối với thiết kế
C1	VRAM hữu hạn	Không nạp draft riêng; giới hạn cache theo độ dài chuỗi.
C2	Chi phí verify	Draft vẫn cần target kiểm chứng; layer-skip cần forward, PLD cần lookup.
C3	Vị trí cache	Đồng bộ logical length, causal mask và RoPE position.
C4	Cách ly trạng thái	Cache speculative chưa xác minh không được thành cache chuẩn.
C5	Sai khác số học	Đo fidelity riêng theo precision, kernel và chế độ thực thi.

Phạm vi kết luận được quy định trong Chương 1 và Chương 5, không xem là ràng buộc kiến trúc. Thiết kế ưu tiên ba nguyên tắc: không nạp thêm trọng số draft, cách ly cache target chuẩn khỏi token chưa xác minh, và đo hiệu năng bằng thời gian toàn chu kỳ. Kiến trúc tổng thể đáp ứng các nguyên tắc này được trình bày ở mục tiếp theo.

3.2 Kiến trúc tổng thể

Hình 1 minh họa các nhánh xử lý: mũi tên xanh là luồng điều khiển, mũi tên liền đen là dữ liệu, nét đứt là tài nguyên dùng chung và nét chấm là đo lường.



Hình 1: Vanilla đi thẳng tới target; draft speculative phải qua bước xác minh.

3.2.1 Các thành phần chính

Prompt/tokenizer chuẩn hóa đầu vào và đặt điều kiện dừng. Các thành phần giải mã còn lại được rút gọn trong Bảng 3.4; các luồng trao đổi thể hiện ở Hình 1.

Bảng 3.4: Các thành phần chính của kiến trúc

Thành phần	**Trách nhiệm chính**
Model adapter	Thống nhất lời gọi model/backend và target forward.
Router/controller	Chọn hành động giải mã và horizon.
Draft provider	Tạo candidates bằng PLD hoặc layer-skip.
Target verifier	Xác minh candidates và quyết định token được giữ.
KV manager	Quản lý cache chuẩn/tạm, commit và rollback.
Profiler	Ghi latency, acceptance, bộ nhớ và diagnostics.

3.2.2 Luồng điều khiển và dữ liệu

Model adapter là lớp dùng chung giữa draft layer-skip và target verifier: layer-skip điều khiển tập lớp được thực thi trong lượt tạo ứng viên, còn verifier gọi target đầy đủ. PLD lấy candidates

từ token history, không cần lượt forward draft. Router đứng trước mỗi hành động decode và chọn phương thức cần chạy; trong trường hợp vanilla, không có candidate draft và hệ thống phát token theo đường target autoregressive. Cả layer-skip và PLD đều chuyển ứng viên cho verifier trước khi cập nhật chuỗi chuẩn.

KV cache manager được dùng xuyên suốt các khối. Cache chuẩn phản ánh tiền tố đã xác minh; cache draft là trạng thái tạm phục vụ tính tiếp ứng viên. Sau verify, manager giữ/commit phần hợp lệ và rollback phần còn lại. Profiler quan sát các pha mà không thay đổi quyết định token: nó ghi thời gian và metrics để so sánh phương pháp trên cùng workload.

Ở mức kiến trúc, một lượt chạy đi qua các bước sau:

1. Tokenizer mã hóa prompt; model adapter chạy prefill và khởi tạo KV cache target chuẩn.

2. Router chọn vanilla, PLD hoặc layer-skip cùng horizon tối đa cho chu kỳ hiện tại.

3. Nếu chọn vanilla, target phát token kế tiếp trực tiếp. Nếu chọn PLD, bộ tra cứu lấy continuation từ n-gram phù hợp. Nếu chọn layer-skip, model adapter chạy đường draft với tập lớp đã cấu hình và cache draft tạm.

4. Khi có candidates, target verifier xử lý chúng theo giao thức giải mã đang dùng; phần hợp lệ được giữ, phần bị từ chối bị loại bỏ và token hiệu chỉnh của target được đưa vào chuỗi.

5. KV cache manager cập nhật trạng thái chuẩn. Profiler ghi thời gian, token phát/được chấp nhận và bộ nhớ; chu kỳ lặp lại đến EOS hoặc giới hạn sinh.

Ranh giới chính giữa các thành phần là chuỗi token cùng trạng thái cache đi kèm. Router chỉ quyết định hành động; draft provider chỉ đề xuất token; verifier là thành phần quyết định token nào được chấp nhận; cache manager bảo đảm các forward tiếp theo bắt đầu từ trạng thái tương ứng với chuỗi đã xác nhận. Chi tiết từng bước draft–verify và các bất biến của cache được trình bày ở mục 3.3 và 3.4.

3.3 Quy trình giải mã suy đoán

Mỗi lần chạy bắt đầu bằng prefill prompt để tạo canonical target KV. Sau đó, hệ thống lặp chu kỳ cho đến khi sinh EOS hoặc đạt ngân sách đầu ra. Nếu router chọn vanilla, target phát token kế tiếp trực tiếp. Với hành động speculative, draft provider tạo candidates; target verifier kiểm tra chúng; cuối cùng hệ thống cập nhật token history và cache theo phần được chấp nhận. Thuật toán 1 mô tả luồng điều khiển tổng quát, không giả định batched NF4 luôn cho sequence identity với vanilla.

Thuật toán 1 Chu kỳ giải mã của ZASSD

Đầu vào: Target M, prompt x, giới hạn sinh B, chính sách router π
Đầu ra: Chuỗi đầu ra y; cache target chuẩn C_{tgt}

$C_{\text{tgt}} \leftarrow \textsc{Prefill}(M, x)$; $y \leftarrow \emptyset$
$p \leftarrow$ token kế tiếp từ logits prefill; $stop \leftarrow$ false ▷ p đang pending, chưa có trong KV
while $|y| < B$ and not $stop$ **do**
  if $p =$ EOS **then**
    append EOS to y; $stop \leftarrow$ true ▷ Không chạy cycle sau EOS
  else
    Đồng bộ trạng thái logic: C_{tgt} ứng với $x \| y$; giữ p làm token đầu vào kế tiếp
    $r \leftarrow B - |y|$; $a, K \leftarrow \pi(x \| y \| p, C_{\text{tgt}})$
    if $a =$ vanilla or $K = 0$ or $r = 1$ **then**
      $q \leftarrow \textsc{TargetForward}(M, p, C_{\text{tgt}})$ ▷ Forward này đưa p vào KV
      append p to y; $C_{\text{tgt}} \leftarrow \textsc{Commit}(C_{\text{tgt}}, p)$; $p \leftarrow q$
    else
      $K \leftarrow \min(K, r - 1)$ ▷ Chừa tối đa một token cho correction/bonus pending
      $d \leftarrow \textsc{Draft}(a, x \| y, p, K, C_{\text{tgt}})$
      if $d = \emptyset$ **then**
        $q \leftarrow \textsc{TargetForward}(M, p, C_{\text{tgt}})$
        append p to y; $C_{\text{tgt}} \leftarrow \textsc{Commit}(C_{\text{tgt}}, p)$; $p \leftarrow q$
      else
        $V \leftarrow \textsc{TargetVerify}(M, [p] \| d, C_{\text{tgt}})$
        $(A, p') \leftarrow \textsc{Resolve}(V, d)$ ▷ A là prefix draft được chấp nhận; p' là correction/bonus
        $E \leftarrow [p] \| A$; cắt E tại EOS hoặc khi đủ ngân sách còn lại
        append E to y; commit KV chỉ cho các token đã được target xử lý trong E
        rollback mọi suffix dự thảo sau điểm commit
        if EOS $\in E$ or $|y| = B$ **then**
          $stop \leftarrow$ true
        else
          $p \leftarrow p'$ ▷ Correction/bonus pending, được xử lý ở cycle tiếp theo
        end if
      end if
    end if
    Ghi token phát ra, token pending, thời gian các pha và logical KV length
  end if
end while
return (y, C_{tgt})

Trong thuật toán, y là các token đã append vào output buffer; p là token kế tiếp do target chọn nhưng đang pending, chưa thuộc y và chưa có KV tương ứng. Vì vậy, cache chuẩn C_{tgt} đại diện cho $x \| y$, còn ngữ cảnh logic dùng để chọn hành động có thể gồm $x \| y \| p$. Khi chọn vanilla, forward kế tiếp nhận p làm đầu vào; khi chọn speculative, p được ghép trước draft candidates trong target verification. Việc đồng bộ token pending với cache được gộp vào

forward hoặc verify đó, không đòi một forward riêng. Correction/bonus p' được chọn ở cuối cycle nhưng chỉ được append và tính vào y trong cycle tiếp theo, nếu còn ngân sách. EOS được tính là một token đầu ra: nếu $p = \text{EOS}$ và còn chỗ, EOS được append rồi quá trình dừng; nếu EOS xuất hiện trong prefix được chấp nhận, phần sau EOS không được phát.

Đặt $r = B - |y|$ là số vị trí đầu ra còn lại tại đầu cycle. Giới hạn $K \leq r - 1$ bảo đảm tối đa $1 + K$ token trong $[p]\|A$ không vượt ngân sách. Chẳng hạn, nếu $r = 4$, $K = 2$ và cả d_1, d_2 được chấp nhận, cycle hiện tại append p, d_1, d_2; token bonus p' vẫn pending và chưa thuộc cache chuẩn. Cycle kế tiếp xử lý rồi append p' vào vị trí còn lại. Vì vậy, p' không được tính vào số token emitted của cycle đã chọn nó.

3.4 Chia sẻ KV cache và các bất biến

Gọi $P = x\|y$ là prompt cùng các token đã append và commit; p là token kế tiếp do target chọn nhưng đang pending. Cache target chuẩn đại diện cho P, chưa chứa p. Ở đầu cycle, p được xử lý cùng forward vanilla hoặc cùng lượt target verification với draft candidates. Sau khi chấp nhận $d_1, \ldots, d_j$, cache chuẩn giữ phần target đã xử lý đến hết prefix $[p, d_1, \ldots, d_j]$; KV của suffix bị từ chối bị crop/rollback. Correction hoặc bonus p' chỉ trở thành token pending cho cycle tiếp theo; nó chưa có KV chuẩn cho đến khi được đưa vào forward target.

Bảng 3.5: Các bất biến của KV cache

Bất biến	**Ý nghĩa**
Prefix correctness	Cache chuẩn đại diện đúng prefix token đã được target xác minh.
Position consistency	Cache length, vị trí token, causal mask và RoPE position đồng bộ.
Rollback isolation	Suffix speculative bị từ chối không còn active trong cache chuẩn.
Draft isolation	Cache draft tạm không làm thay đổi logical state chuẩn.

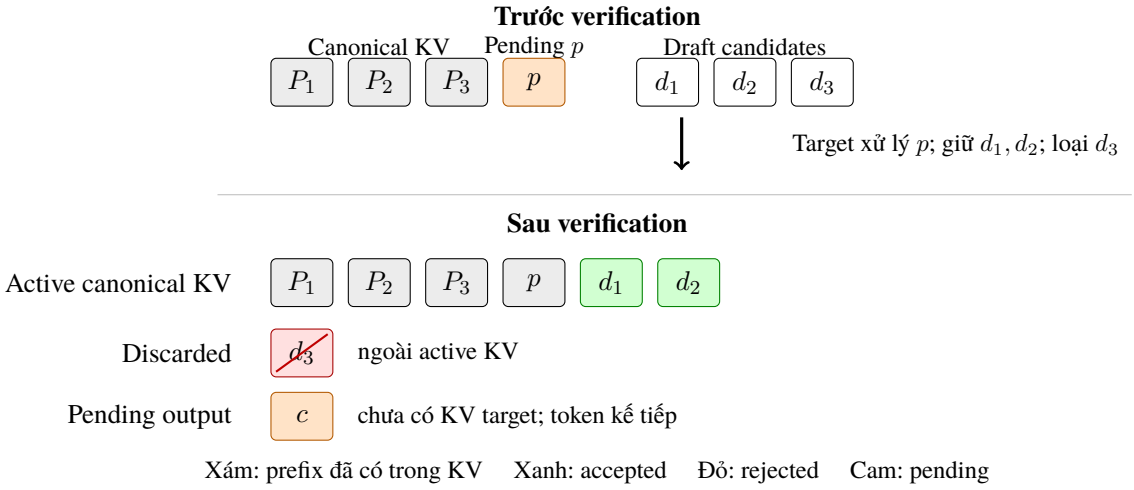
Runtime kiểm tra các bất biến bằng logical-length/position checks sau commit và rollback, so sánh tensor KV/logits với reference theo tolerance đã đặt, và replay token trajectory trên cùng prefix. Đây là ba mức đo riêng; logical consistency không tự chứng minh numerical equivalence hay sequence identity. Backend static có thể chia sẻ buffer vật lý nhưng phải duy trì logical length chuẩn và draft độc lập.

Ba tiêu chí này có quan hệ nhưng không thay thế cho nhau:

Bảng 3.6: Phân biệt ba mức đúng đắn cần đánh giá

Thuộc tính	**Ý nghĩa**
Logical consistency	Cache chỉ đánh dấu active những vị trí và lịch sử token hợp lệ theo quy trình commit/rollback.
Numerical equivalence	KV/logits sai khác reference không quá ngưỡng số học đã định.
Sequence identity	Toàn bộ output token IDs trùng với vanilla trên cùng prompt và cấu hình giải mã.

Logical consistency không bảo đảm numerical equivalence hay sequence identity; hai tiêu chí này cần đo riêng.



Hình 2: Trạng thái KV trước và sau verification: target xử lý pending input p, chấp nhận d_1, d_2, từ chối d_3 và chọn correction c. Sau commit/rollback, active KV chứa $P\|p\|d_1\|d_2$; c được xử lý ở forward target kế tiếp.

Trong Hình 2, $P = x\|y$ là prefix chuẩn trước cycle; p là token pending đầu vào, chưa có trong KV lúc bắt đầu verification. Target xử lý p, chấp nhận d_1, d_2 và loại d_3. Ký hiệu c trong hình chính là token pending p' của thuật toán. Trạng thái sau verify là:

$$H_{\text{logical}} = P\|p\|d_1\|d_2\|c \quad (c \text{ đang pending}),$$
$$C_{\text{tgt}} = \text{KV}_{\text{target}}(P\|p\|d_1\|d_2).$$

Trong đó H_{logical} gồm lịch sử đã commit cùng token kế tiếp đã được chọn; c chưa được đưa vào KV chuẩn cho đến khi target xử lý nó ở forward kế tiếp.

3.5 Mô hình chi phí và điều kiện hòa vốn

Ranh giới cycle trong công thức khớp với một vòng lặp giải mã của implementation. N_{emit} là số token được append vào output buffer trong vòng đó, sau khi áp dụng EOS và giới hạn token còn lại; đây là emitted count được ghi cho cycle. Token pending p cũ được tính khi vòng hiện tại append nó; correction/bonus p' mới chỉ được tính trong vòng sau nếu được append. T_{cycle} là wall-clock của chính vòng lặp ấy, từ lúc bắt đầu xử lý hành động đến khi sẵn sàng bắt đầu vòng tiếp theo; nó gồm chi phí draft hoặc lookup, verify/forward target, cập nhật cache, điều phối và đồng bộ cần thiết. Vì vậy, forward xử lý p cũ và tạo prediction p' mới nằm trong cùng khoảng thời gian, còn N_{emit} chỉ đếm các token append trong khoảng đó. T_v là latency trung bình trên mỗi token của vanilla, đo trên cùng model, backend, prompt/workload, thiết bị, thiết lập warmup và chế độ sinh với phương pháp cần so sánh. Speedup throughput dài hạn được ước lượng:

$$S \approx \frac{\mathbb{E}[N_{\text{emit}}]T_v}{\mathbb{E}[T_{\text{cycle}}]}. \tag{3.1}$$

Để phân rã thời gian, các mốc pha phải dùng khoảng đo không chồng lấn và tổng thời lượng các khoảng khớp với wall-clock cycle. Nếu profiler ghi các pha lồng nhau hoặc bất đồng bộ, không cộng trực tiếp các số đó; dùng wall-clock cycle làm mẫu số, còn pha chồng lấn báo cáo riêng để chẩn đoán. Khi đó, một phân rã không chồng lấn có thể viết:

$$T_{\text{cycle}} = T_{\text{draft}} + T_{\text{verify}} + T_{\text{cache}} + T_{\text{lookup}} + T_{\text{control}} + T_{\text{sync}}, \tag{3.2}$$

Trong Hình 3, $T_{\text{other}} = T_{\text{control}} + T_{\text{sync}}$. Hình nhóm các pha để diễn giải chi phí, không phải kết quả profiling. Một số thành phần có thể bằng không hoặc không áp dụng cho hành động cụ thể; không cộng lại chi phí đã nằm trong một khoảng đo bao hàm khác. Trong profiling, tốc độ dự đoán của hành động a tại quan sát z_t được ước lượng theo cùng tỷ số token phát ra trên thời gian chu kỳ:

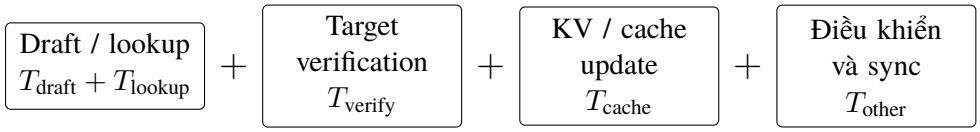
$$\widehat{\text{TPS}}(a \mid z_t) = \frac{\widehat{\mathbb{E}}[N_{\text{emit}} \mid a, z_t]}{\widehat{\mathbb{E}}[T_{\text{cycle}} \mid a, z_t]}. \tag{3.3}$$

Đây là mô hình dự đoán dùng cho router; nó không bảo đảm dự báo chính xác ngoài cohort hiệu chuẩn. Chương 5 báo cáo phép đo thực nghiệm thay vì coi công thức xấp xỉ là kết quả hiệu năng.

Điều kiện hòa vốn theo kỳ vọng so với vanilla là:

$$\mathbb{E}[T_{\text{cycle}}] < \mathbb{E}[N_{\text{emit}}]T_v. \tag{3.4}$$

Với layer-skip, T_{draft} là forward qua các lớp không bị bỏ và có thể tương đối lớn; với PLD, forward draft được thay bằng lookup cost nhưng số candidates phụ thuộc vào n-gram match của workload. T_{verify} phụ thuộc horizon K, chiều dài ngữ cảnh và backend/kernel. Điều kiện hòa vốn là mô hình throughput dài hạn theo kỳ vọng, không bảo đảm speedup cho từng prompt hoặc từng cycle; acceptance rate cao riêng lẻ cũng không chứng minh đạt hòa vốn.



Hình 3: Các thành phần thời gian khái niệm trong một speculative cycle.

3.6 Thiết kế bộ định tuyến

Tại chu kỳ t, router triển khai chọn giữa vanilla, PLD và layer-skip, kèm horizon ứng viên. Router dùng ước lượng throughput tại Công thức (3.3) để so sánh các phương án. Trong code hiện tại, quyết định không phải argmax thuần trên mọi hành động: router dùng cost estimates, baseline vanilla, confidence/acceptance EMA hoặc predictor nếu được cấp, n-gram match và một margin ưu tiên PLD; các gate có thể chọn vanilla khi dự đoán layer-skip không vượt baseline. Cấu hình mặc định bật giới hạn horizon cho hit yếu và hiệu chỉnh horizon PLD dài. Quy tắc tổng quát có thể mô tả:

$$a_t \in \{\text{vanilla}, \text{PLD}, \text{layer-skip}\}, \qquad a_t = \text{Policy}_{\text{implemented}}(z_t) \tag{3.5}$$

trong đó z_t gồm thông tin mà nhánh triển khai nhận được ở thời điểm gọi: entropy target, logit margin nếu bật, lịch sử/đầu ra dò PLD, thống kê acceptance gần đây, cost profile và latency tracker tùy cấu hình.

Với đường hybrid router dùng trong CLI, entropy, match length, số candidates, acceptance EMA/predictor và latency draft/verify từ cost profile hoặc latency tracker tham gia ước lượng. Telemetry GPU như VRAM/nhiệt độ không phải đặc trưng trực tiếp của quyết định tại đường gọi này; không mô tả chúng như tín hiệu router đang dùng. Các giá trị acceptance và thời gian là dự báo từ profile/thống kê, không bảo đảm acceptance của cycle kế tiếp.

Router chọn vanilla khi PLD miss và layer-skip bị gate vì thấp hơn baseline dự đoán, hoặc khi speculative action không khả dụng. Khi có PLD hit, chính sách so throughput PLD

với layer-skip có margin ưu tiên PLD; ở PLD-only mode, PLD phải đạt vanilla gate. Do đó argmax là diễn giải mục tiêu ước lượng chứ không phải mô tả đầy đủ từng nhánh quyết định. Hiệu quả phải được đánh giá bằng benchmark paired với baseline cố định mạnh nhất; chính sách không chứng minh rằng chọn động luôn tốt hơn.

Hiện thực chương trình

Chương này mô tả cách các thành phần thiết kế ở Chương 3 được tổ chức và gọi trong mã nguồn ZASSD. Trọng tâm là ranh giới giữa decoder, target verifier, cache và backend; chương không lặp lại mô hình draft–verify hay trình bày kết quả tốc độ. Các cấu hình và số liệu benchmark được báo cáo ở Chương 5.

Lưu ý về phiên bản mã nguồn. Các Chương 3–5 mô tả hệ thống ZASSD ở giai đoạn đầu của đồ án (vanilla, layer-skip, PLD, router và lớp ứng dụng). Sau đó kho mã được tập trung lại vào giải mã exact trên NF4 (Chương 6): các thành phần layer-skip, router, ứng dụng và dữ liệu thực nghiệm của giai đoạn đầu đã được gỡ khỏi nhánh làm việc hiện tại. Bản nguyên vẹn của hệ thống mô tả ở đây nằm tại git tag `evidence-pre-cleanup` (commit `06cf0d7`); cấu trúc thư mục ở Hình 4 là của bản đó.

4.1 Môi trường và tổ chức mã nguồn

ZASSD được cài đặt bằng Python, dùng PyTorch và giao diện mô hình causal language model của Hugging Face Transformers. Cấu hình thực nghiệm chính là Qwen2.5-3B-Instruct trên NVIDIA GeForce RTX 4050 Laptop GPU. Tệp kiểm tra môi trường trong repository ghi nhận Python 3.12.13, PyTorch 2.11.0+cu128, CUDA runtime 12.8 và Transformers 5.14.1; GPU có 6141 MiB bộ nhớ báo cáo bởi driver. Các phiên profiling dùng NVIDIA Nsight Systems và Nsight Compute.

Bảng 4.1: Môi trường triển khai và công nghệ sử dụng

Thành phần	**Cấu hình hoặc vai trò**
Ngôn ngữ	Python 3.12.13.
Framework	PyTorch 2.11.0+cu128; Transformers 5.14.1.
Mô hình chính	Qwen2.5-3B-Instruct; revision được ghi trong manifest của từng phép đo khi có áp dụng.
GPU	NVIDIA GeForce RTX 4050 Laptop GPU; 6141 MiB theo thông tin thiết bị.
Driver NVIDIA	595.99.02 theo manifest của một phép đo cụ thể.
Quantization	bitsandbytes NF4 trong đường benchmark CLI; HQQ W4A16 + GemLite trong đường kiểm chứng backend riêng.
Profiling	NVIDIA Nsight Systems để xem timeline CUDA/NVTX; Nsight Compute để khảo sát kernel.

Ví dụ provenance của một phép đo trong manifest `tradeoff_n25_r1/environment.json`: model revision bắt đầu bằng `aa8e7253`, commit mã bắt đầu bằng `6c3ddf7f`, NVIDIA driver `595.99.02`. Mã SHA đầy đủ được lưu trong manifest; các giá trị trên chỉ mô tả artifact này.

Các package được chia theo nhiệm vụ thay vì đặt toàn bộ logic trong CLI. Hình 4 cho thấy cấu trúc thư mục đang có trong repository; các script benchmark và dữ liệu thực nghiệm nằm ngoài package chính.

```
src/zassd/
|-- cli.py
|-- decoding/
|-- baselines/
|-- cache/
|-- models/
|-- layer_selection/
|-- routing/
|-- profiling/
|-- backends/
|-- controllers/
|-- app/
`-- utils/
```

Hình 4: Cấu trúc package thực tế dưới `src/zassd`.

4.2 Hiện thực các phương pháp giải mã

4.2.1 Vanilla Decoding

Đường baseline thực hiện prefill toàn bộ prompt, lấy argmax hoặc lấy mẫu từ logits cuối, rồi đưa token vừa chọn vào lượt forward kế tiếp cùng KV cache. Sinh dừng khi gặp EOS hoặc đủ `max_new_tokens`. Trong benchmark CLI, hàm `run_vanilla` dùng `TargetKVCache` dạng static và đồng bộ CUDA trước/sau khoảng đo. Module `src/zassd/decoding/vanilla.py` cung cấp hàm sinh và các trường đo như số token, thời gian token đầu, TPOT, throughput và peak VRAM cho các luồng gọi khác.

Vanilla là baseline cùng target model và prompt. Khi chạy benchmark một phương pháp suy đoán, CLI cũng đo vanilla trên cùng danh sách prompt để tính throughput tương đối; warmup được chạy trước phần đo và không tính vào kết quả.

4.2.2 Layer-Skip Self-Speculative Decoding

Trong `src/zassd/decoding/speculative.py`, hàm `self_speculative_generate` điều khiển prefill và các cycle. Ở pha draft, `LayerManager` nhận danh sách chỉ số lớp cần bỏ qua. Thay vì xóa layer khỏi `ModuleList`, manager tạm thay `forward` của từng lớp được chọn bằng ánh xạ identity, giữ nguyên thứ tự và chỉ số lớp của mô hình. Sau lượt tạo candidates, các hàm gốc được phục hồi. Cách làm này tránh nạp thêm checkpoint draft và giữ các chỉ số lớp tương thích với cache của mô hình.

Danh sách lớp có thể đến từ preset của CLI hoặc được truyền trực tiếp cho API. Bộ chọn trong `src/zassd/layer_selection/` hỗ trợ chiến lược static, random, empirical và CKA; CLI hiện khai báo các preset `mid_12`, `mid_10`, `mid_8`, `empirical_6/8/10`, `cka_75` và `cka_50`. Các thí nghiệm riêng có thể dùng danh sách lớp khác; vì vậy tên preset không thay thế cho việc lưu chính xác chỉ số lớp trong manifest.

Việc thay `forward` chỉ diễn ra trong context draft của `LayerManager.skip_layers`; khối `finally` gọi `restore_layers` kể cả khi phần sinh draft phát sinh ngoại lệ. Lượt target verification chạy sau khi context kết thúc, do đó dùng lại các hàm `forward` gốc. Các lớp bị bỏ qua không tự cập nhật KV trong draft; những lớp còn hoạt động vẫn dùng chỉ số lớp và vị trí logic của cache gốc. Điều này chỉ bảo toàn cách định vị cache, không chứng minh KV của draft tương đương với KV target. Hàm identity hiện trả lại `hidden_states` (hoặc tuple có các phần phụ là `None`), nên phụ thuộc vào kiểu đầu ra của kiến trúc đang hỗ trợ; không thể áp dụng mặc nhiên cho layer có side effect hoặc yêu cầu đầu ra cache/attention riêng.

Draft layer-skip bắt đầu từ token pending và cache draft tạm, sinh tối đa K ứng viên, sau đó chuyển quyền quyết định sang target đầy đủ. Chỉ target verifier xác định token nào được

giữ. Phần này mô tả luồng thực thi; acceptance, fidelity và tốc độ theo từng cấu hình được dành cho Chương 5.

4.2.3 Prompt Lookup Decoding

PLD được hiện thực trong `src/zassd/baselines/prompt_lookup.py`; thao tác khớp n-gram nằm trong `src/zassd/decoding/ngram.py`. Bộ tìm kiếm thử hậu tố lịch sử với độ dài từ `ngram_size` giảm dần đến 2 token. Khi có nhiều match ở cùng độ dài, mã chọn continuation của lần xuất hiện gần nhất. Nguồn tra cứu có thể là toàn bộ history, prompt, hoặc prompt kết hợp phần đã sinh; cấu hình mặc định của API dùng history. Số token continuation trả về bị giới hạn bởi horizon yêu cầu.

Nếu lookup miss hoặc không có token continuation, vòng sinh chuyển sang bước vanilla cho token kế tiếp. Nếu tìm thấy candidates, target xác minh chúng theo thứ tự và PLD dùng cùng quy trình commit/crop cache như đường speculative. Confidence do hàm tra cứu trả về được tạo từ độ dài match, số lần lặp và mức đồng thuận continuation; đây là heuristic cho router, không phải xác suất acceptance đã hiệu chuẩn. Module cũng có chế độ buộc miss để làm đối chứng và chế độ sequential verification dành cho kiểm tra greedy; chế độ sequential không nhận temperature khác 0.

4.2.4 Target Verification

Verifier greedy trong `src/zassd/decoding/verification.py` so sánh từng candidate với argmax của target tại vị trí tương ứng. Các candidate trùng khớp liên tiếp tạo thành accepted prefix. Tại sai khác đầu tiên, candidate đó và phần suffix sau nó bị từ chối, còn token argmax của target trở thành correction. Nếu tất cả K candidate được chấp nhận, argmax ở vị trí kế tiếp là bonus token. Với speculative decoder, các logits được lấy từ forward target qua token pending đầu vào cùng candidates; vì vậy target đồng thời xử lý token pending trước khi xác minh dãy draft.

Phần token được append vào output trong cycle được phân biệt với correction/bonus đang pending. Token pending được đưa vào KV và chuỗi output ở forward/verification tiếp theo nếu ngân sách còn. Khi cycle chạm EOS hoặc hết ngân sách, thuật toán dừng mà không phát thêm token vượt giới hạn. Các kiểm tra numerical trace, cache replay và margin fallback là chế độ chẩn đoán tùy chọn; chúng không được xem như chi phí mặc định của lượt benchmark thông thường.

4.3 Hiện thực KV cache và chọn lớp

`src/zassd/cache/kv_cache.py` cung cấp `TargetKVCache` với hai lựa chọn: static cache tự cấp phát và `DynamicCache` của Transformers. Static cache cấp buffer key/value theo lớp ở lần cập nhật đầu, sau đó chép trạng thái mới vào lát cắt kế tiếp và theo dõi `seq_len` logic. Khi reject, `crop` giảm độ dài active; dữ liệu nằm sau độ dài này không còn được coi là prefix hợp lệ. Dynamic cache tăng theo cơ chế của Transformers và cũng hỗ trợ crop về prefix cần giữ.

Draft cache được tạo từ canonical target cache qua `fork_ephemeral_draft_kv`. Với static cache, các tensor key/value nền được chia sẻ nhưng đối tượng cache draft giữ độ dài logic riêng. Với cache động, các layer cache draft khởi đầu bằng tham chiếu tới tensor prefix; cập nhật và crop được quản lý tách khỏi trạng thái độ dài canonical. Sau target verification, cache chuẩn được crop tới độ dài prefix cộng với token đầu vào đã xử lý và accepted prefix. Nhờ vậy, rejected suffix không còn active trong các forward tiếp theo.

Trong static cache, draft chia sẻ các buffer key/value đã cấp phát, nhưng chỉ được ghi từ vị trí `seq_len` trở đi; prefix có chỉ số nhỏ hơn độ dài canonical được dùng chung như vùng chỉ đọc. Target verification sau đó ghi KV target vào các vị trí ứng viên tương ứng. Suffix bị ghi thử có thể vẫn còn dữ liệu vật lý sau rollback, nhưng `crop` chỉ hạ logical length để các vị trí đó không còn thuộc active prefix; thao tác này không giải phóng buffer GPU đã cấp phát. Hai kiểm tra tương ứng trong `tests/test_kv_cache_invariants.py` là `test_static_invariant_2_draft_writes_do_not_mutate_prefix` (so sánh K/V prefix trước và sau draft write) và `test_static_invariant_3_target_verification_and_rollback` (kiểm tra logical length sau rollback).

Không nạp draft weights, chia sẻ buffer KV và giảm tổng VRAM sử dụng là ba thuộc tính khác nhau. Layer-skip loại bỏ nhu cầu về một bộ trọng số draft riêng; static draft state có thể dùng lại buffer key/value; nhưng target weights, cache chuẩn, activation, workspace kernel và bộ đệm preallocated vẫn tiêu thụ VRAM. Do đó, dung lượng cấp phát của cache và peak VRAM toàn tiến trình được đo riêng, không suy ra từ tên phương pháp.

Bảng 4.2 liên kết một số module với trách nhiệm cụ thể và vị trí kiểm tra liên quan.

Bảng 4.2: Mapping giữa package mã nguồn và chức năng triển khai

Module	Trách nhiệm	Trạng thái/kiểm tra liên quan
decoding/speculative	Điều khiển cycle, budget, draft và verify.	Ghi token phát, acceptance, cycle stats và logical KV length.
decoding/verification	Chấp nhận prefix greedy và tạo correction/bonus.	Có test verification và speculative exactness.
prompt lookup / n-gram	Tra cứu candidate và fallback khi miss.	Có test match, fallback và sequential verify.
cache KV / draft state	Cache chuẩn, fork draft, crop/rollback và metadata draft.	Có test invariants, độ dài và tương đương sau rollback.
model adapter / layer manager	Truy cập layer và áp dụng logical layer skip.	Kiểm tra chỉ số layer và phục hồi forward gốc.
layer selection	Chọn danh sách lớp static, empirical hoặc CKA.	Lưu danh sách skip cụ thể cùng cấu hình chạy.

4.4 Router và thu thập dữ liệu

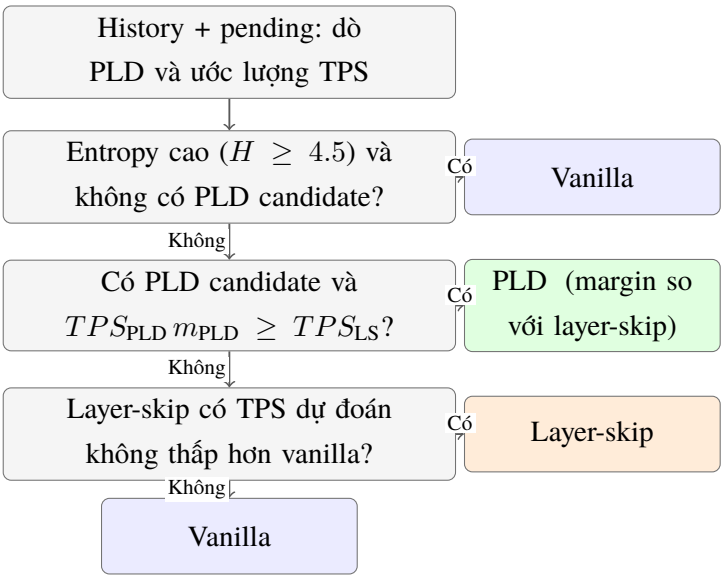
`src/zassd/routing/hybrid_router.py` nhận history, token pending, horizon và entropy; logit margin chỉ được truyền khi cấu hình cần nó. Router dò PLD để xác định candidates và độ dài match. Acceptance EMA/predictor cùng cost profile hoặc latency tracker ước lượng chi phí và số token có thể được giữ. Đây là dự báo từ profile hay quan sát trước đó, không phải acceptance đã biết của cycle hiện tại.

Các gate có thể chọn vanilla nếu layer-skip được dự báo chậm hơn baseline, hoặc khi entropy cao mà PLD không có candidates. Khi PLD có match, router so sánh throughput dự báo với layer-skip và áp dụng margin ưu tiên PLD; sau PLD miss, layer-skip có thể được chọn nếu qua gate. Telemetry GPU không phải tín hiệu trực tiếp của router trong đường gọi CLI hiện tại. Chính sách và công thức ước lượng được trình bày ở Mục 3.6; bảng sau tóm tắt tín hiệu được nối vào code.

Bảng 4.3: Tín hiệu và vai trò trong router hiện thực

Tín hiệu	**Cách sử dụng trong code**
N-gram match	Xác định PLD candidates, nguồn và độ dài match.
Acceptance EMA/predictor	Ước lượng số draft token được chấp nhận; predictor là tùy chọn.
Cost profile / latency tracker	Ước lượng thời gian draft, verify và vanilla theo action/horizon.
Entropy	Điều chỉnh acceptance dự báo và hỗ trợ gate fallback.
Logit margin	Đặc trưng bổ sung khi predictor được bật; không bắt buộc ở mọi cấu hình.
PLD preference margin	Điều chỉnh quyết định PLD so với layer-skip khi cả hai khả dụng.

Hình 5 tóm tắt nhánh quyết định của router khi layer-skip và `enable_vanilla_skip` được bật. Trong code hiện tại, `ENTROPY_PANIC_THRESHOLD=4.5`; gate này có thể bị tắt bằng cấu hình. TPS trong sơ đồ là giá trị dự đoán từ profile/EMA, không phải acceptance đã biết trước.



Hình 5: Các gate router theo thứ tự thực thi. Nhánh PLD có thể chọn draft trước gate so sánh với vanilla.

Đối tượng `SpeculativeMetrics` ghi tổng token, thời gian prefill/draft/verify/cache, thời gian điều khiển và phần thời gian còn lại; đồng thời lưu acceptance, số verification cycle, số cycle PLD/layer-skip và peak VRAM. Với diagnostic tracing, mỗi cycle có thể ghi candidate IDs, token accepted/emitted, token pending kế tiếp, cache length trước/sau và margin logits.

Tùy chọn NVTX đánh dấu các vùng prefill, draft, verify và fallback để Nsight Systems định vị thời gian trên timeline. Nsight Compute được dùng ở các phép đo kernel riêng; số đo kernel có overhead profiler và không thay thế latency đầu-cuối.

Có một khác biệt cần lưu ý khi đọc gate: nhánh PLD được xét trước gate so sánh layer-skip với vanilla. Nếu PLD có candidate và $TPS_{\text{PLD}} \times \texttt{prefer_pld_margin} \geq TPS_{\text{LS}}$ theo dự đoán, router chọn PLD mà không so tiếp với vanilla; do đó PLD vẫn có thể được chọn dù TPS dự đoán thấp hơn baseline. Test trong `tests/test_hybrid_router.py` ghi nhận chính trường hợp này. Sơ đồ thể hiện thứ tự điều kiện của implementation hiện tại, không phải chính sách tối ưu hóa vanilla toàn cục.

CLI benchmark lưu tóm tắt gồm tên model/phương pháp, K, strategy, vanilla throughput, throughput phương pháp, speedup và acceptance trung bình; mỗi prompt có throughput, số token, acceptance và speedup. JSON của CLI này không ghi peak VRAM hoặc toàn bộ phase timings. Các giá trị đó có trong metrics ở cấp Python hoặc artifacts của benchmark/profiler chuyên dụng. Phần kết quả tương ứng được trình bày ở Chương 5.

4.5 Tích hợp inference backend

Đường CLI chính nạp causal LM qua Transformers. Mặc định, model được lượng tử hóa 4-bit NF4 bằng bitsandbytes với phép tính FP16; tùy chọn `--no-quantize` nạp model FP16. Module `src/zassd/backends/base.py` bổ sung loader có tên backend để các benchmark kiểm soát so sánh nhận cùng giao diện model/tokenizer. Loader này hiện được gọi trong script backend validation riêng, chưa được nối thành tùy chọn backend trong lệnh `zassd benchmark`.

Bảng 4.4: Khác biệt triển khai giữa hai đường backend được khảo sát

Thuộc tính	Transformers + BNB NF4	HQQ + GemLite W4A16
Định dạng	NF4 4-bit; compute FP16.	W4A16; group size 64, axis 1 theo loader hiện có.
Nạp model	Transformers quantization config và device map.	Nạp model FP16, sau đó GemLite patch các linear layer theo processor HQQ.
Giao diện decoder	Là đường mặc định của benchmark CLI và các decoder Hugging Face.	Trả model tương thích API Transformers qua loader backend riêng.
Phụ thuộc	bitsandbytes trong môi trường chính.	hqq và gemlite là phụ thuộc tùy chọn; cần phiên bản tương thích.
Đường chạy	CLI `benchmark`; có thể tắt lượng tử hóa để chạy FP16.	Script backend validation gọi `load_backend`; chưa có cờ chọn trực tiếp trong CLI benchmark.
Đánh giá fidelity	So token/cache với vanilla trong cùng backend và cấu hình.	Cũng cần baseline vanilla riêng của backend; không suy ra sequence identity giữa hai định dạng.

Việc đổi backend có thể thay đổi kernel, bộ nhớ trung gian, phép tính KV/logits và kết quả token. Vì vậy, BNB NF4 và HQQ+GemLite được xem là hai cấu hình triển khai riêng; báo cáo không giả định backend nào luôn nhanh hơn, dùng ít VRAM hơn hoặc tạo cùng chuỗi token. Các kết luận định lượng được giới hạn theo protocol ở Chương 5.

4.6 Giao diện chạy và kiểm tra

Entry point `zassd` được khai báo trong `pyproject.toml`. Sau khi cài package ở chế độ editable, các lệnh dưới đây sử dụng tên subcommand và tham số được khai báo trong parser của `src/zassd/cli.py`. Benchmark mặc định dùng NF4; hai lần chạy ghi kết quả vào hai tệp riêng để đối chiếu:

```
python -m pip install -e .
mkdir -p results

zassd benchmark --model Qwen/Qwen2.5-3B-Instruct \
  --method vanilla --prompt "Explain how a cache works." \
  --max-new-tokens 32 --output results/vanilla.json
```

```
zassd benchmark --model Qwen/Qwen2.5-3B-Instruct \
  --method prompt_lookup --k 4 \
  --prompt "Explain how a cache works." \
  --max-new-tokens 32 --output results/pld.json
```

Hai lệnh trên minh họa cách gọi CLI. Trong workspace biên soạn báo cáo này, lệnh CLI và test cache chưa chạy được vì môi trường Python không có PyTorch; log lần thử được lưu tại `experiments/chapter4_validation/current_workspace_attempt.log`. Để cung cấp kết quả chạy thực tế có thể đối chiếu, Bảng 4.5 trích hai dòng từ phép đo paired benchmark đã lưu trong repository. Đây là kết quả của harness backend validation, không phải JSON do lệnh `zassd benchmark` tạo ra.

Bảng 4.5: Trích kết quả paired benchmark thực tế: vanilla và PLD trên cùng prompt

Phương pháp	**Decode (token/s)**	**Acceptance**	**Đối chiếu vanilla**
Vanilla, BNB NF4	37.16	–	Mốc tham chiếu.
PLD $K = 4$, BNB NF4	33.45	21.1%	Không khớp toàn bộ; divergence đầu tiên ở token 23.

Phép đo dùng prompt `qasper_119_ctx384` (384 token), Qwen2.5-3B-Instruct revision bắt đầu `aa8e7253`, đầu ra tối đa 32 token, NF4, SDPA và một lượt đo sau warmup 32 token. Execution commit là `4d5e54f9`; đây là pilot một prompt, không phải kết luận hiệu năng tổng quát.

Hai artifact được lưu dưới `experiments/backend_validation/paired_benchmark/`:

```
bnb_ctx384_32_fullwarm.jsonl
fullwarm_32_summary.json
```

Bảng 4.6: Trạng thái kiểm thử được xác minh trong workspace hiện tại

Kiểm tra	**Kết quả**
Router (9 test)	9 passed; chạy standalone vì `tests/conftest.py` yêu cầu PyTorch.
KV cache invariants	Chưa chạy được: pytest dừng ở bước import `torch` trong `tests/conftest.py`.
CLI benchmark	Chưa chạy được: `zassd.cli` dừng ở bước import `torch`; không tạo output mới.

Các lệnh và cấu trúc JSON mô tả giao diện sử dụng. Kết quả benchmark được lưu cùng cấu hình và metadata; trạng thái test trong bảng trên phản ánh đúng các lệnh đã chạy ở workspace hiện tại.

Như vậy, phần hiện thực dùng chung target model cho baseline và các nhánh speculative, đồng thời tách rõ backend loading, candidate generation, target verification và quản lý cache. Mã nguồn lưu các cấu hình chạy cùng số liệu cần thiết để Chương 5 phân tích hiệu năng và fidelity theo đúng backend, horizon, workload và chế độ thực thi.

Phương pháp thực nghiệm và kết quả

Chương này đánh giá ZASSD theo ba khía cạnh: khả năng tái lập phép đo, hiệu năng so với giải mã vanilla và mức độ bảo toàn đầu ra. Các kết quả được chia theo từng cohort. Benchmark paired 150 prompt là nguồn kết quả hiệu năng chính; các phép đo router, phân tích số học và profiling có giao thức riêng nên chỉ được dùng để trả lời câu hỏi tương ứng, không gộp vào trung bình của benchmark chính.

5.1 Thiết lập thực nghiệm

5.1.1 Cấu hình benchmark chính

Bảng 5.1 tóm tắt cấu hình ghi trong manifest của benchmark paired. Các phép đo này được thực thi tại commit `a37a4e4`; revision mô hình, tokenizer, phiên bản thư viện và hash dữ liệu được lưu cùng artifact để có thể kiểm tra lại.

Bảng 5.1: Cấu hình của benchmark paired chính

Nhóm	**Cấu hình được ghi nhận**
Phần cứng	NVIDIA GeForce RTX 4050 Laptop GPU; các snapshot nhiệt độ trong phiên chạy dao động 83–88 °C. Benchmark chính không lưu thông tin CPU/RAM như một phần của manifest.
Hệ điều hành	Linux 6.8.0-94-generic, glibc 2.39; Python 3.12.13.
Framework	PyTorch 2.11.0+cu128, CUDA 12.8, Transformers 5.14.1, bitsandbytes 0.50.0.
Mô hình	Qwen2.5-3B-Instruct, 36 lớp; model và tokenizer revision `aa8e72537993ba99e69dfaafa59ed015b17504d1`.
Backend	bitsandbytes NF4, tính toán FP16, attention SDPA.
Sinh token	Greedy decoding; tối đa 48 token đầu ra; cùng prompt và target model giữa các phương pháp.
Mã nguồn	Commit thực thi `a37a4e4cda647a5f5fc2b6861fb1d28f7db48eea`.

Thông tin phần cứng và phần mềm không hoàn toàn giống nhau giữa mọi thí nghiệm trong repository. Vì vậy, Bảng 5.1 chỉ áp dụng cho benchmark chính; những cohort dùng backend, context length hoặc chế độ tracing khác được mô tả riêng tại mục tương ứng.

5.2 Tập dữ liệu và giao thức đánh giá

Benchmark chính gồm 150 prompt: 50 mẫu GSM8K, 50 mẫu HumanEval và 50 mẫu CNN/DailyMail. Mỗi prompt được chạy với giới hạn 48 token sinh. Giao thức gồm hai prompt warmup cho mỗi tập, một repeat và ba seed xoay vòng thứ tự phương pháp (42, 43, 44). Thiết kế này tạo 450 nhóm prompt/thứ tự paired và 2.700 dòng quan sát thô. Mỗi phương pháp được so sánh trên cùng prompt; khi tính khoảng tin cậy, các repeat và seed thứ tự được lấy trung bình trong từng prompt trước khi bootstrap theo prompt.

Thời gian tiền xử lý và sinh được tách thành prefill, thời gian đến khi có logits prompt cuối cùng (TTFT), và decode sau prefill. Trong artifact này, token đầu được chọn từ logits prefill; `decode_tokens` không tính token đó, còn end-to-end TPS tính cả prefill. Hai đối chứng kiểm tra overhead cũng được giữ lại: vanilla chạy qua speculative core và PLD bị buộc lookup miss. Cả hai đối chứng giữ nguyên token sequence nhưng chậm hơn direct vanilla, cho thấy có chi phí của đường thực thi/harness; kết quả speedup không được hiệu chỉnh cộng bù overhead này.

Bảng 5.2: Các cohort thực nghiệm và phạm vi sử dụng

Cohort	**Quy mô/cấu hình**	**Vai trò trong báo cáo**
Benchmark paired chính	150 prompt; NF4; tối đa 48 token	Kết quả throughput, speedup và exact-match của PLD, PLD gate, router; là căn cứ cho kết luận hiệu năng chính.
Router paired	30 prompt; NF4; static KV; tối đa 32 token; K PLD = 2, 4, 8	Phân tích router trên cohort prompt ngữ cảnh ngắn; có development/held-out, không gộp với benchmark chính.
Numerical fidelity	30 prompt chuẩn bị sẵn; NF4; K=4; tối đa 32 token	So sánh SDPA/eager, static/dynamic và verification tuần tự; thời gian có tracing chỉ mang tính chẩn đoán.
Nsight profiling	Một prompt dài 384 token; tối đa 32 token	Quan sát kernel và API trong phiên có instrumentation; không dùng như phép đo production throughput.

5.3 Phương pháp đối chứng và chỉ số

Các phương pháp trong benchmark chính gồm direct vanilla; *vanilla via speculative core* và PLD forced-miss làm đối chứng overhead; PLD với $K = 4$; PLD gate chọn giữa PLD và vanilla; và router chọn giữa PLD ($K \leq 4$), layer-skip ($K = 2$) hoặc vanilla. Do đó, đây là so sánh giữa các phương thức có cùng target model, prompt và cấu hình decode trong một harness, chứ không phải phép đo mọi phương pháp trên mọi backend.

Với prompt i, throughput end-to-end ở một lượt chạy là số token đầu ra chia cho tổng thời gian gồm prefill và decode. Gọi $r_{m,i,r,o}$ là tỷ số throughput của phương pháp m với vanilla trên cùng prompt i, cùng repeat r và seed thứ tự o:

$$r_{m,i,r,o} = \frac{\text{TPS}_{m,i,r,o}}{\text{TPS}_{\text{vanilla},i,r,o}}, \qquad \bar{r}_{m,i} = \frac{1}{|\mathcal{R}_i|} \sum_{(r,o)\in\mathcal{R}_i} r_{m,i,r,o}, \qquad S_m = \frac{1}{N} \sum_{i=1}^{N} \bar{r}_{m,i}. \tag{5.1}$$

Do đó, S_m là trung bình các tỷ số paired ở cấp prompt sau khi gộp các lượt đo của chính prompt đó; nó không phải tỷ số giữa mean TPS của hai phương pháp. CI 95% được tạo bằng bootstrap prompt: lấy mẫu lại N prompt có hoàn lại, tính lại S_m cho mỗi mẫu bootstrap và lấy hai phân vị 2.5%/97.5%. Con số PLD $1.063\times$ trong Bảng 5.3 được lấy theo đúng định nghĩa này từ summary/statistics của artifact. Decode TPS là số token decode chia cho thời gian decode và không đồng nhất với end-to-end TPS. Acceptance rate là tỷ lệ draft token được target chấp nhận trên tổng số draft token được đưa ra xác minh. Exact sequence match là tỷ lệ prompt mà toàn bộ token ID đầu ra trùng với direct vanilla. Các chỉ số exactness không suy ra từ acceptance rate.

Peak VRAM trong router evaluation được ghi bằng hàm `max_memory_allocated` của PyTorch chia cho 1024^2, tức MiB bộ nhớ tensor đã cấp phát cực đại; đây không phải NVML process memory hay bộ nhớ reserved. Tên trường artifact là `peak_vram_mb`, nhưng đơn vị quy đổi thực tế là MiB. Chỉ so sánh trực tiếp trong cùng cohort/cùng phương thức thu thập. Khi báo cáo numerical fidelity, KV drift, logit drift, argmax agreement và sequence identity được xem là các đại lượng riêng: có sai khác tensor không nhất thiết làm đổi token; ngược lại, một token khác làm mất exact sequence match dù phần lớn chuỗi vẫn trùng.

5.4 Kết quả benchmark chính

5.4.1 So sánh paired trên ba tập dữ liệu

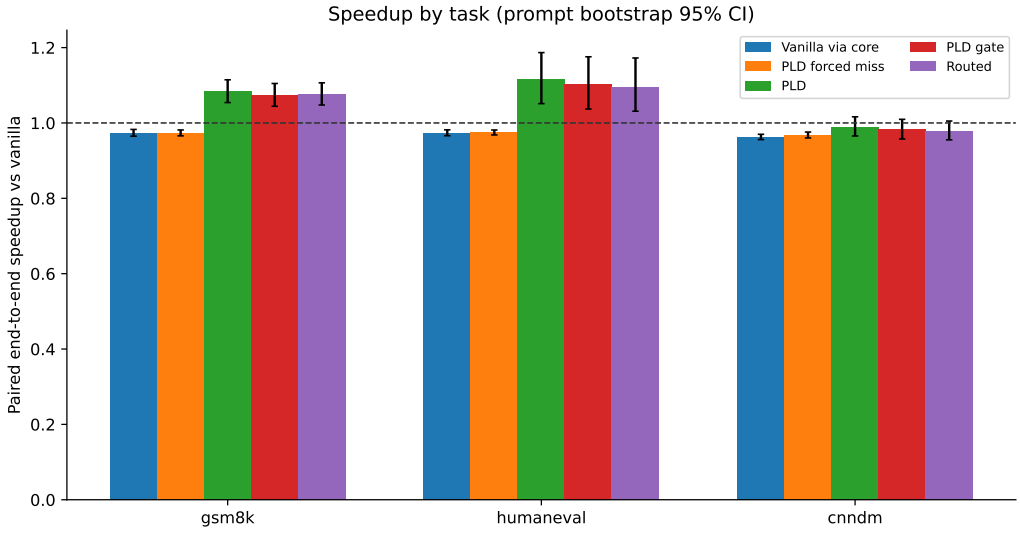
Bảng 5.3 trình bày kết quả end-to-end speedup từ `experiments/benchmark_validation/full_n50_r1_a37a4e4/summary.json`. CI 95% được bootstrap theo prompt, không coi ba thứ tự chạy của cùng prompt là ba prompt độc lập.

PLD đạt speedup trung bình 6.3% theo tỷ số throughput paired; CI bootstrap không chứa 1 trong cohort này. Tuy nhiên, phân bố theo prompt không đồng đều: median speedup PLD là 1.007, phân vị 10–90 là 0.928–1.258, và 53.3% prompt nhanh hơn vanilla. Theo từng tập, speedup PLD lần lượt là 1.084 trên GSM8K, 1.115 trên HumanEval và 0.989 trên CNN/DailyMail. Kết quả này cho thấy lợi ích không xuất hiện đồng đều theo workload.

Bảng 5.3: Speedup end-to-end và exact sequence match trong benchmark chính

Phương pháp	**Speedup (95% CI)**	**Prompt nhanh hơn vanilla**	**Exact match**
Vanilla trực tiếp	1.000	–	100.0%
Vanilla qua speculative core	0.970 [0.965, 0.975]	14.0%	100.0%
PLD forced miss	0.972 [0.968, 0.976]	13.3%	100.0%
PLD, $K = 4$	**1.063 [1.036, 1.092]**	53.3%	83.3%
PLD gate	1.053 [1.025, 1.081]	48.7%	83.3%
Router	1.050 [1.023, 1.079]	49.3%	83.3%

Router đạt 1.050× so với vanilla, nhưng tỷ số throughput paired router/PLD là 0.989 (CI 95%: 0.985–0.993). Như vậy router chậm hơn PLD trong giao thức đã đo, dù vẫn nhanh hơn vanilla ở mức trung bình. PLD gate cũng thấp hơn PLD khoảng 0.9% theo tỷ số paired. Không nên diễn giải router là cải thiện phương pháp cố định tốt nhất chỉ vì nó có speedup lớn hơn 1 so với vanilla.

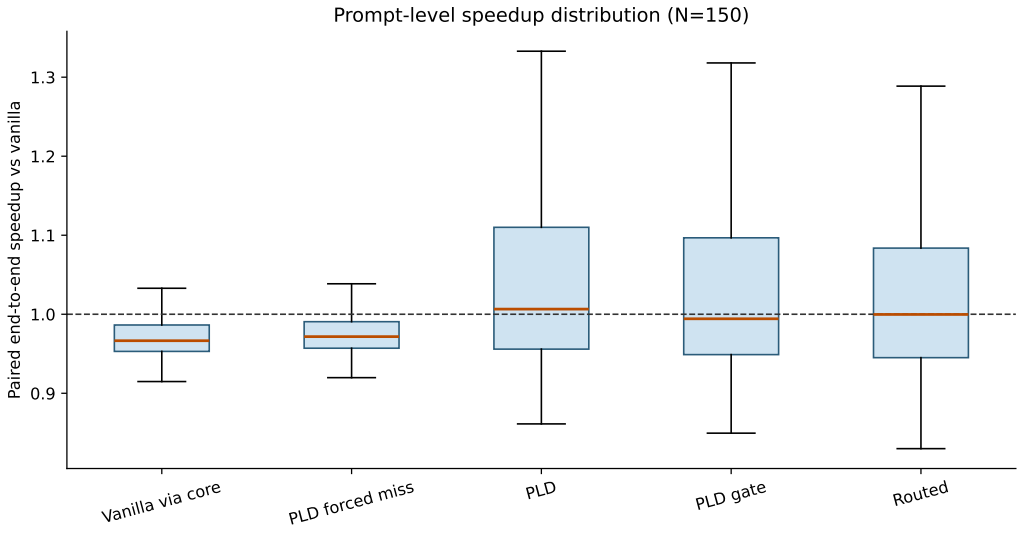


Hình 6: Speedup end-to-end của các phương pháp theo từng benchmark trong cohort 150 prompt.

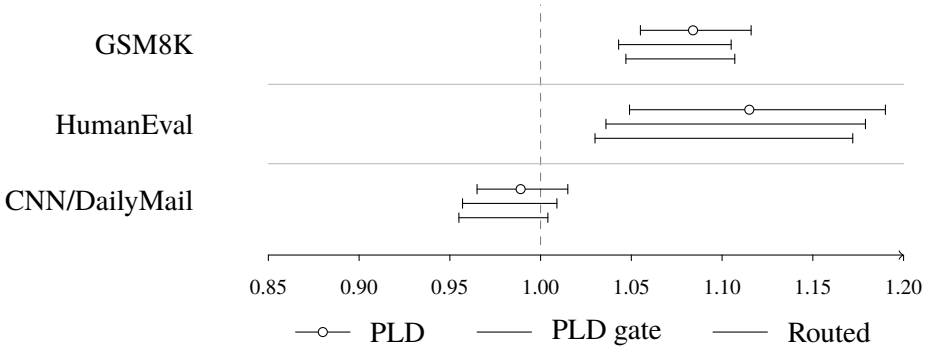
Để làm rõ phân bố ở cấp prompt, Hình 7 trình bày boxplot speedup paired cho 150 prompt. Trung bình PLD lớn hơn 1 nhưng trung vị chỉ xấp xỉ 1; chỉ 53.3% prompt nhanh hơn vanilla. Vì vậy, trung bình tổng hợp không có nghĩa PLD cải thiện trên đa số workload với biên độ đồng đều.

Hình 8 phóng to vùng quanh hòa vốn bằng trục tung thu hẹp từ 0.85 đến 1.20. Các điểm và khoảng tin cậy theo từng tập được vẽ đơn sắc, phân biệt bằng ký hiệu để dễ đọc khi in trắng đen. Khoảng tin cậy là bootstrap theo prompt trong từng tập.

Các negative control vanilla-via-core và PLD-forced-miss chậm hơn vanilla trực tiếp lần



Hình 7: Phân bố speedup end-to-end paired theo prompt; đường đứt đoạn biểu thị hòa vốn $1.0\times$.



Hình 8: Speedup theo tập trên vùng tập trung 0.85–$1.20\times$; trục tung được thu hẹp để thấy rõ chênh lệch quanh đường hòa vốn $1.0\times$. Ký hiệu đơn sắc: hình tròn PLD, hình vuông PLD gate, tam giác router; thanh ngang là CI bootstrap 95%.

lượt khoảng 3.0% và 2.8%. Đây là overhead quan sát được của đường thực thi trong harness, không phải một hiệu chỉnh để cộng vào kết quả PLD. Việc giữ control giúp phân biệt lợi ích do lookup trúng với chi phí nền của speculative path.

5.4.2 Layer-skip và router trên cohort riêng

Trong đánh giá router độc lập gồm 30 prompt ngữ cảnh 384 token, CKA90 với $K = 2$ đạt 0.902× vanilla (CI 95%: 0.875–0.929), trong khi hybrid router đạt 1.045× (1.001–1.090) và PLD $K = 2$ được chọn từ development đạt 1.095× (1.054–1.139). Trên 14 prompt held-out, CKA90 đạt 0.917× (0.880–0.956), router 1.078× (1.022–1.142), PLD $K = 2$ 1.134× (1.081–1.194). Đây là cohort prepared-input ngắn, một lượt mỗi prompt; nó cho thấy cấu hình layer-skip được khảo sát chậm hơn vanilla, nhưng không thể dùng để kết luận mọi cách chọn lớp hoặc mọi context đều chậm.

5.5 Phân tích chi phí và điều kiện hòa vốn

Speculative decoding chỉ có lợi khi thời gian hoàn thành một cycle đủ thấp so với số token thực sự được phát ra trong cycle đó. Với thời gian vanilla trung bình mỗi token T_v, điều kiện hòa vốn có thể viết $\mathbb{E}[T_{\text{cycle}}] < \mathbb{E}[N_{\text{emit}}]T_v$. Vì vậy acceptance cao tự nó không chứng minh tăng throughput: một draft tốn thời gian hoặc target verification đắt có thể xóa hết lợi ích của nhiều token được chấp nhận. Mô hình ở Chương 3 được dùng như quan hệ giải thích; phép đo phải tránh cộng chồng các khoảng thời gian đồng thời.

$$T_{\text{cycle}} = T_{\text{draft}} + T_{\text{verify}} + T_{\text{cache}} + T_{\text{lookup}} + T_{\text{control}} + T_{\text{sync}}, \tag{5.2}$$

Trong đó các thành phần chỉ được cộng trực tiếp khi được đo trên các khoảng không chồng lấn; các tổng thời gian API host và kernel GPU của profiler không thể cộng cơ học với nhau. Tại benchmark chính, PLD có speedup nhưng router và PLD gate thấp hơn PLD; điều này phù hợp với việc quyết định/fallback bổ sung có chi phí, song benchmark tổng thể không đủ để quy toàn bộ chênh lệch cho riêng một thành phần.

Artifact benchmark lưu số token decode, tổng số verification cycle, số cycle PLD/layer-skip, accepted tokens và tổng decode wall time. Từ các bộ đếm này có thể suy ra một số thống kê gộp như Bảng 5.4. Các giá trị là tỷ số tổng trên 450 lượt prompt/order của mỗi phương pháp, không phải trung bình không trọng số của từng cycle.

Đây là thống kê mô tả gộp được suy ra từ bộ đếm tổng; artifact không lưu thời gian độc lập cho từng cycle. Vì vậy, các giá trị này giúp kiểm tra quan hệ tổng thể giữa số token, số cycle và thời gian, nhưng không xác định chi phí thành phần hay nguyên nhân thời gian của

từng cycle.

Bảng 5.4: Thống kê cycle gộp suy ra từ raw benchmark paired

Phương pháp	Accepted tokens per spec cycle	Decode tokens per verify cycle	Decode time per cycle (ms)	Vanilla fallback cycles
PLD $K = 4$	1.134	1.217	31.82	78.5%
Router	1.125	1.227	32.51	77.5%

Số candidate draft được đề xuất mỗi cycle không được lưu trong raw file, nên không thể tính trực tiếp đại lượng này hoặc acceptance fraction trên tổng candidates từ artifact hiện tại. Số cycle không draft là phần dư giữa tổng verification cycles và số PLD/layer-skip cycles, dựa trên các trường đếm của harness; vì vậy được xem là thống kê suy ra. Thời gian mỗi cycle được tính bằng tổng decode wall time chia tổng verification cycles. Với vanilla baseline, decode wall time gộp tương ứng khoảng 27.19 ms/token. Thay các thống kê gộp vào $\mathbb{E}[N_{\text{emit}}]T_v/\mathbb{E}[T_{\text{cycle}}]$ cho decode-only ratio khoảng 1.041 cho PLD và 1.026 cho router; các tỷ số này khớp với pooled decode TPS tính từ cùng token/time totals theo định nghĩa, nên là kiểm tra nhất quán số học chứ không phải dự đoán độc lập. Chúng cũng không thay thế end-to-end paired speedup ở mục 5.4 vì công thức này không tính prefill.

Để khảo sát chi phí ở mức kernel, một phép Nsight riêng chạy trên cùng một prompt 384 token, NF4/FP16, SDPA và giới hạn 32 token. Bảng 5.5 tổng hợp thời gian kernel GPU (tổng các kernel có thể chồng lấn) và tổng thời gian API host cho đồng bộ/phát lệnh. Đây là số liệu có instrumentation, không phải end-to-end latency production.

Bảng 5.5: Một số số đo Nsight chẩn đoán trên một prompt cố định

Phương pháp	Tổng GPU kernel (ms)	NF4 GEMM (ms)	Dequant (ms)	Sync API (ms)
Vanilla	828.2	312.8	66.5	378.1
PLD $K = 4$	811.1	400.0	54.3	270.6
CKA90 $K = 2$	951.7	451.7	40.0	376.8

Số liệu cho thấy thành phần NF4 GEMM, dequant và các lời gọi đồng bộ đều hiện diện đáng kể trong trace. Tuy nhiên, tổng GPU kernel có thể bao gồm overlap, còn synchronize API là thời gian host-side; chúng không phải các thành phần độc lập của critical path. Do đó, từ trace này chỉ có thể mô tả hoạt động quan sát được, chưa thể kết luận dequant hoặc synchronization là nguyên nhân duy nhất gây chậm. Manifest và các tệp xuất Nsight được lưu tại `experiments/backend_validation/kernel_profile/`.

5.6 Đánh giá bộ nhớ và tính đúng đắn

5.6.1 Bộ nhớ

Các phương pháp khảo sát không nạp thêm checkpoint draft, nhưng điều đó không đồng nghĩa với không phát sinh thêm VRAM: vẫn có KV cache, activation, vùng làm việc kernel và buffer đã cấp phát. Trong router evaluation 30 prompt (một cohort khác benchmark chính), peak allocated memory trung bình ghi nhận khoảng 2 107 MiB cho direct vanilla, 2 110 MiB cho PLD $K = 2$, 2 111 MiB cho PLD $K = 4$, 2 112 MiB cho CKA90 $K = 2$, và 2 113 MiB cho hybrid router. Đây là giá trị trung bình của `torch.cuda.max_memory_allocated`, không phải VRAM NVML/reserved. Chênh lệch nhỏ trong phép đo này không thể khái quát sang mọi context length, backend hoặc cơ chế cấp phát; đây cũng không phải số liệu peak VRAM của cohort 150 prompt.

5.6.2 Numerical fidelity và sequence identity

Benchmark paired chính có exact sequence match 83.3% cho PLD, PLD gate và router. Phân tích can thiệp trên chính nhóm 25 prompt mismatch (7 GSM8K, 5 HumanEval, 13 CNN/DailyMail) ghi nhận vanilla lặp lại ổn định; trace mặc định có khác biệt digest active KV trước token mismatch, còn verification tuần tự canonical khớp vanilla đủ 48 token trên các prompt này. Kết quả là bằng chứng mạnh cho sự phụ thuộc đường tính toán/cache trong cấu hình NF4 đã thử, nhưng không chứng minh cùng tỷ lệ divergence ở precision hoặc backend khác.

Một kiểm chứng riêng trên 30 prompt, giới hạn 32 token và $K = 4$, phân chia 16 prompt chẩn đoán và 14 held-out cho thấy ảnh hưởng của backend/chế độ thực thi. Mười sáu ID chẩn đoán được lấy từ cohort PLD $K = 4$ ngữ cảnh ngắn đã ghi nhận mismatch trong phép chạy độc lập trước đó; chúng được chọn để khảo sát nguyên nhân sai khác, không lấy ngẫu nhiên từ traffic. Mười bốn ID held-out là các mẫu còn lại trong cùng prepared-input artifact và không được dùng khi chọn nhóm chẩn đoán, nhưng cũng không phải một tập dữ liệu ngoài độc lập. Bảng 5.6 báo cáo sequence match và các chỉ số drift tách biệt.

Trong các hàng batched held-out, drift KV/logit được ghi nhận nhưng token sequence vẫn trùng vanilla; đây là ví dụ trực tiếp cho thấy numerical equivalence và sequence identity không tương đương. Mức 0/16 exact match của SDPA batched trên nhóm chẩn đoán là kết quả có điều kiện trên một cohort được chọn vì từng có mismatch, không phải ước lượng tỷ lệ lỗi cho workload tự nhiên. Tương ứng, 14/14 exact match held-out chỉ mô tả 14 mẫu chưa chọn trong cùng artifact; cỡ mẫu nhỏ và nguồn mẫu chung không cho phép kết luận xác suất lỗi thực tế bằng 0. Ở nhóm chẩn đoán, chuyển sang verification tuần tự khôi phục exact match

Bảng 5.6: Kết quả numerical fidelity trên cohort 30 prompt; thời gian có tracing chỉ mang tính chẩn đoán

Cấu hình	Cohort	Exact match	KV/logit drift	Token divergence
SDPA, static, batched	Diagnostic (16)	0/16	16/16	16/16
SDPA, static, batched	Held-out (14)	14/14	14/14	0/14
Eager, static, batched	Diagnostic (16)	8/16	16/16	8/16
Eager, static, batched	Held-out (14)	14/14	14/14	0/14
SDPA, dynamic, batched	Diagnostic (16)	0/16	16/16	16/16
SDPA, dynamic, batched	Held-out (14)	14/14	14/14	0/14
SDPA, static, tuần tự	Diagnostic (16)	16/16	0/16	0/16
SDPA, static, tuần tự	Held-out (14)	14/14	0/14	0/14

16/16, đồng thời làm TPS chẩn đoán giảm so với cấu hình batched. Các con số thời gian tại đây có tracing snapshot overhead nên không được so với benchmark production ở mục 5.4.

5.6.3 Kiểm tra logic KV cache

Các test về prefix không bị draft ghi đè, target verification và rollback được mô tả tại Chương 4 và nằm trong `tests/test_kv_cache_invariants.py`. Chúng kiểm tra invariant ở mức cấu trúc cache/logical length, trong khi cohort numerical fidelity ở trên đánh giá sai khác tensor và token trên các prompt cụ thể. Hai nhóm bằng chứng bổ sung cho nhau: test invariant không tự bảo đảm logits hoặc chuỗi token luôn tương đương vanilla. Kết quả kiểm thử chỉ nên gắn với log chạy tương ứng; không suy rộng sang các cấu hình chưa chạy.

5.7 Thảo luận và giới hạn

Bảng 5.7 tóm tắt câu trả lời thực nghiệm trong phạm vi các cohort hiện có.

Các giới hạn chính gồm: benchmark hiệu năng chính chạy trên một GPU và một cấu hình NF4/SDPA; tốc độ biến thiên theo prompt và tập dữ liệu; exact sequence match thấp hơn 100% trong phương pháp suy đoán; các phép router, fidelity và profiling dùng cohort/cấu hình riêng; profiling có instrumentation; và chưa có cơ sở để khái quát sang mô hình, GPU, precision hay backend khác. Các thí nghiệm context/backend exploratory cũng không được trộn vào kết quả xác nhận của benchmark 150 prompt.

Trong điều kiện GPU VRAM hạn chế, một kỹ thuật speculative decoding đáng sử dụng khi đồng thời thỏa ba điều kiện có thể kiểm chứng: (1) tổng chi phí cycle đủ thấp so với số token được phát ra, thay vì chỉ có acceptance cao; (2) nó cải thiện trên baseline cố định mạnh nhất đã đo, không chỉ vượt vanilla; và (3) mức speedup đáp ứng yêu cầu numerical fidelity/sequence identity của ứng dụng. Trong thí nghiệm này, PLD vượt vanilla trung bình

Bảng 5.7: Tổng hợp kết quả theo câu hỏi nghiên cứu

Câu hỏi	**Kết luận có giới hạn theo bằng chứng**
RQ1	PLD tăng tốc trung bình trên cohort 150 prompt (1.063×), nhưng hiệu quả phụ thuộc tập dữ liệu; CNN/DailyMail đạt 0.989×. CKA90/K2 chậm hơn vanilla trong cohort router 30 prompt đã thử.
RQ2	Draft, verify, lookup, điều khiển và đồng bộ cùng ảnh hưởng thời gian cycle. Nsight cho thấy các hoạt động GEMM, dequant và sync, nhưng phép profiling đơn lẻ chưa xác định critical path nhân quả. Acceptance cao không bảo đảm tăng tốc.
RQ3	Với NF4, batched execution có các trường hợp KV/logit drift và token divergence; verification tuần tự khớp vanilla trên cohort fidelity đã thử nhưng chậm hơn trong phép đo có tracing. Kết luận không thay thế kiểm chứng trên precision/backend khác.
RQ4	Router tăng tốc so với vanilla trong benchmark chính (1.050×) nhưng thấp hơn PLD (tỷ số paired 0.989; CI 0.985–0.993). Router hiện chưa chứng minh lợi ích so với fixed PLD trong giao thức này.

nhưng có mismatch token và không nhanh hơn trên CNN/DailyMail; router vẫn dưới PLD; verification tuần tự khôi phục sequence identity trong cohort fidelity nhưng tốn thời gian hơn. Vì vậy, lựa chọn thực tế phụ thuộc vào workload, ngân sách latency và yêu cầu chính xác đầu ra.

Giải mã suy đoán exact trên mô hình NF4 không dùng thêm bộ nhớ GPU

Chương này trình bày phần nghiên cứu mở rộng sau các phép đo ở Chương 5: thay vì chấp nhận rằng speedup của PLD đi kèm đầu ra có thể lệch khỏi vanilla, chương xây dựng một vòng giải mã suy đoán *exact* cho mô hình lượng tử hóa NF4 và đánh giá nó với giao thức tách riêng tập phát triển và hai cohort held-out dùng đúng một lần. Toàn bộ kết quả là bằng chứng thực nghiệm hữu hạn trên một GPU, chưa phải chứng minh.

6.1 Động lực và mục tiêu

Giải mã suy đoán tăng tốc suy luận bằng cách đề xuất vài token tương lai rồi kiểm tra chúng song song trong một lần chạy mô hình [2, 3]. Với greedy, đầu ra *về nguyên tắc* bằng đầu ra giải mã tuần tự. Trên thực tế, "bằng nhau" chỉ đúng nếu lần chạy theo khối cho logits và KV giống hệt bước một-token. Các toán tử trong mô hình chọn kernel theo hình dạng đầu vào, nên đầu ra theo khối có thể chỉ gần đúng và quỹ đạo có thể rẽ nhánh; Mục 6.6.2 cho thấy toán tử nào thực sự gây lệch trong ngăn xếp bitsandbytes 0.50.0 và PyTorch SDPA. Trong giao thức trước của kho mã này, Prompt Lookup Decoding (PLD) [4] trên NF4 [20] chỉ khớp đầu ra không suy đoán ở 83.3% chuỗi ($N{=}150$) và $21/60$ ở một phép so sánh backend sau đó.

Mục tiêu. Cho một tham chiếu chính xác (greedy bitsandbytes M=1 không sửa đổi), xây dựng vòng giải mã suy đoán (i) cho đúng chuỗi token tham chiếu, (ii) không dùng mô hình draft và không dùng thêm bộ nhớ GPU đáng kể, (iii) nhanh hơn tham chiếu. Chúng tôi đặt tính đúng đắn lên trước tốc độ: tốc độ chỉ có nghĩa khi (i) đã giữ.

Đóng góp.

1. Một vòng giải mã exact B2 và các thành phần của nó (Mục 6.3), kèm cách đọc tính đúng đắn theo toán tử và phân loại rõ phần nào theo cấu trúc mã, phần nào chỉ có bằng chứng thực nghiệm (Mục 6.4).

2. Hai phát hiện: (i) trong ngăn xếp bitsandbytes 0.50.0 và PyTorch, kiểm tra theo khối làm lệch quỹ đạo chủ yếu qua attention và LM head chứ không qua phép chiếu NF4 (Mục 6.6.2); (ii) mã nguồn CUDA của bitsandbytes đọc theo nghĩa đen cộng tích lũy tuần tự, nhưng bản nhị phân thật khớp thứ tự cộng theo cặp của kernel Triton (Mục 6.4.1).
3. Bằng chứng nhiều tầng: token và KV bitwise từng bước, quay lui, EOS, ngữ cảnh tới 3071 token, ba kích cỡ mô hình, hai cohort held-out đăng ký trước (Mục 6.6).
4. Các kết quả bất lợi và âm có ghi chép: baseline biên dịch nhanh hơn, hướng cải thiện không hiệu quả (Mục 6.7).

Công trình gần nhất chúng tôi biết là LLM-42 [16], dùng vòng decode–verify–rollback để đảm bảo xác định ở mức hệ thống phục vụ trên FP16 nhiều GPU; chúng tôi giải một bài toán khác ở mức số học trong lượng tử hóa 4-bit trên một GPU tiêu dùng.

6.2 Bài toán và định nghĩa

Cố định trọng số, trạng thái lượng tử hóa, phiên bản thư viện, kiến trúc GPU, cài đặt attention, thủ tục prefill, kiểu và dung lượng cache, quy tắc chọn token khi bằng nhau và quy tắc EOS. Gọi $R(S_t, x_t) = (z_t, S_{t+1})$ là bước chuyển một-token tham chiếu: chạy `model(x)` với bitsandbytes không sửa đổi, x có hình dạng $(1, 1)$. Trạng thái S_t gồm mọi thứ ảnh hưởng tới lần chạy sau (mọi mục KV, độ dài logic, vị trí). *Exact* nghĩa là chuỗi token sinh ra bằng chuỗi do R sinh, và (ở các kiểm tra mạnh hơn) KV cũng bằng từng bit tại mọi bước.

Chúng tôi phân biệt hai tham chiếu. **B1** là greedy với lịch hình dạng cố định (mọi phép toán được đệm về cùng số hàng vật lý); nó làm đổi số học của mô hình nên không bằng greedy gốc. **B2** là greedy bitsandbytes gốc; chỉ B2 là mục tiêu của chương này.

6.3 Phương pháp

Mỗi chu kỳ gồm: tra n-gram để đề xuất tối đa $k{=}2$ token; chạy mô hình trên $1{+}$#đề xuất hàng; chấp nhận tiền tố khớp dài nhất; quay lui KV phần bị loại. Các thành phần:

Kernel NF4 $M{=}3$ (tùy chọn, chỉ để tăng tốc). `_fused_nf4_m3_kernel` (Triton [21]) xử lý ba hàng activation với ba bộ cộng dồn độc lập, cùng một trọng số giải lượng tử dùng chung. Mỗi lane cộng 32 tích fp16 (`mul.rn.f16x2`), tích lũy trong fp32 theo từng cặp, rồi gộp 32 lane bằng cây dịch $16, 8, 4, 2, 1$. Chạy với `enable_fp_fusion=False`. Mã không có câu lệnh nào đọc dữ liệu của hàng khác nên đầu ra từng hàng chỉ phụ thuộc hàng đó. Kernel này *không cần cho tính exact*: trong bitsandbytes 0.50.0 các phép chiếu NF4 chuẩn đã cho đầu ra từng

Thuật toán 2 Một lượt sinh của B2 (rút gọn)

Đầu vào: prompt, ngân sách N, k=2

prefill bằng BNB gốc; *pending* ← token đầu của greedy
while chưa đủ N token và **chưa** gặp EOS **do**
    $C \leftarrow$ PLD(lịch sử + *pending*, $\leq k$ token)
    if $C = \emptyset$ **then** ▷ quay về bước gốc
        chạy BNB M=1 trên *pending*; *pending* ← argmax
    else
        hoán đổi sang bản vá; chạy decoder trên $[pending] + C$ (một lần)
        for $i = 0, 1, \ldots$ **do** ▷ LM head lười
            $p_i \leftarrow$ argmax lm_head(h_i); **nếu** $i < |C|$ **và** $p_i \neq C_i$ **thì** dừng
        end for
        phát *pending* và các C_j được chấp nhận; cắt KV về độ dài đã commit
        pending ← $p_{\text{hàng cuối đã tính}}$
    end if
end while

hàng bằng bước M=1 (Mục 6.6.2), và B2 lười không dùng kernel Triton vẫn khớp BNB ở $10/10$ prompt dev (token và KV). Kernel chỉ giúp tăng tốc: $1.161\times$ so với $1.117\times$ khi dùng phép chiếu chuẩn (10 prompt GSM8K dev).

Phép toán từng hàng. RMSNorm, LM head và attention được gọi riêng cho từng hàng: attention gọi `scaled_dot_product_attention` một lần cho mỗi hàng với K/V cắt đúng $L_t{+}i{+}1$ vị trí, không mặt nạ, `enable_gqa=True`, nên hình dạng giống lời gọi một-token.

Quay lui và quay về M=1. Cache tĩnh chỉ trả về $[0, \text{seq_len})$ nên cắt độ dài logic là đủ để các mục dư không bao giờ được đọc trước khi bị ghi đè. Khi không có đề xuất, vòng chạy đúng bước M=1 gốc (không lịch, không kernel Triton) bằng cách hoán đổi `forward` từng module giữa bản gốc và bản đã vá; lịch được vào một lần cho cả lượt sinh.

LM head lười. Chiếu LM head cho từng hàng chỉ cho tới hàng bị loại đầu tiên: các hàng sau không bao giờ được dùng và không ghi trạng thái, nên bỏ qua chúng không đổi kết quả.

6.4 Lập luận về tính đúng đắn

Gọi *tính bằng nhau theo hàng* (RE) là: với khối bắt đầu từ trạng thái bằng tham chiếu, hàng i của mọi tensor trung gian bằng từng bit với tensor tương ứng của R áp dụng lần lượt $x_0, \ldots, x_i$, và KV ghi tại vị trí $L_t{+}i$ cũng bằng. Từ RE, quy nạp theo chu kỳ cho kết quả: (1) logits hàng i bằng tham chiếu; (2) tiền tố được chấp nhận và *pending* bằng chuỗi greedy; (3) cắt độ dài

Bảng 6.1: Trạng thái lập luận theo toán tử. **[S]**: theo cấu trúc mã; **[E]**: chỉ có bằng chứng thực nghiệm; **[O]**: còn mở.

Toán tử	Cơ sở	Nhãn
Embedding, ghi KV, argmax, LM head	gọi từng hàng hoặc sao chép byte	[S]
RMSNorm	gọi riêng từng hàng, giống tham chiếu	[S]
Attention	một lời gọi SDPA mỗi hàng, cùng hình dạng, bố cục	[S] nếu dispatch chỉ phụ thuộc hình dạng/stride; [O]
Rotary, residual, SiLU·mul	phép toán theo phần tử; chưa kiểm riêng bản vector hóa	[S]/[O]
Phép chiếu NF4 (252)	bitsandbytes chuẩn: 0 lệch trong 9.9×10^8 phần tử khi gọi ba hàng so với từng hàng; kernel Triton tùy chọn có ba bộ cộng dồn độc lập	[E]; [S]+[E] cho kernel Triton

logic giữ lại đúng $[0, L_t{+}j{+}1)$; (4) chu kỳ không đề xuất chính là R. Giả thiết duy nhất không thuộc cấu trúc là RE ở từng toán tử.

6.4.1 Thứ tự rút gọn của kernel bitsandbytes

Phần này chỉ liên quan tới kernel Triton tùy chọn: nó cho biết khi nào kernel đó thay được kernel bitsandbytes mà không đổi kết quả. Mã nguồn bitsandbytes 0.50.0 (`kgemm_4bit_inference_naive`) cộng tuần tự từng tích fp16 trong mỗi lane, còn kernel Triton của chúng tôi cộng từng cặp tích liền kề trước. Theo số học IEEE hai thứ tự này khác nhau. Chúng tôi mô phỏng cả hai thứ tự trong fp32 trên $28\,672\,000$ đầu ra ngẫu nhiên đuôi nặng (năm lớp, đủ bảy phép chiếu). Kernel bitsandbytes thật và kernel Triton *đều khớp mô phỏng theo cặp ở* $28\,672\,000/28\,672\,000$ *đầu ra* và đều khác mô phỏng tuần tự ở 398 đầu ra; phép thử đủ nhạy để phân biệt hai thứ tự. Cận trên 95% của tỷ lệ lệch mỗi đầu ra khoảng 1.0×10^{-7} (quy tắc ba). Chúng tôi chưa giải thích được vì sao bản nhị phân khác mã nguồn đã đọc, nên mọi lần nâng cấp bitsandbytes phải chạy lại phép mô phỏng này.

6.5 Thiết lập thực nghiệm

Môi trường. Qwen2.5-3B-Instruct [22], revision `aa8e7253...`, bitsandbytes 0.50.0 (NF4, lượng tử hóa kép, tính toán fp16), Triton 3.6.0, PyTorch 2.11.0, transformers 5.14.1, một GPU RTX 4050 Laptop 6 GB, batch 1, greedy, $k{=}2$, PLD với n-gram ≤ 3, 128 token sinh nếu không ghi khác.

Bảng 6.2: Bằng chứng tính đúng đắn của B2 (so với greedy bitsandbytes gốc).

Kiểm tra	Quy mô	Kết quả
Held-out GSM8K 300–349 (token)	50 prompt	$50/50$
Held-out GSM8K 400–449 (token và KV từng bước)	50 prompt	$50/50$ và $50/50$
Dev GSM8K, HumanEval (token)	60 prompt	$60/60$
Ngữ cảnh 512–3071 token (token và KV)	12 prompt	$12/12$
Kết thúc bằng EOS, sinh tới 384 token (token và KV)	18 prompt	$18/18$
Quay lui/loại tại các vị trí $0, 1$, nhiều chu kỳ (3 prompt trên 3B, 1 prompt mỗi mô hình 0.5B và 1.5B)	5 kịch bản	đạt
Qwen2.5-0.5B và 1.5B (token và KV)	10 prompt mỗi mô hình	$10/10$
Phép chiếu NF4 trên activation thật	252 module, 9.29M phần tử	bằng từng bit

Dữ liệu và giao thức. Tập *dev* (GSM8K, HumanEval, CNN/DM) dùng để phát triển và chọn cấu hình. Hai cohort *held-out* GSM8K, chỉ số test 300–349 và 400–449, được đóng băng theo chỉ số công khai, kiểm tra không trùng ID/prompt với mọi benchmark cục bộ, đăng ký cấu hình trước khi chạy và chỉ dùng *một lần*; chúng không được dùng để chọn cấu hình. Việc này không loại trừ dữ liệu công khai đã lọt vào dữ liệu huấn luyện mô hình.

Chỉ tiêu. (i) Token khớp tham chiếu BNB eager; (ii) băm SHA256 của KV tại mọi tiền tố không-terminal; (iii) tốc độ $=$ thời gian BNB eager $/$ thời gian phương pháp, trung vị trên 2–3 vòng xoay thứ tự, trung bình nhân giữa các prompt kèm khoảng tin cậy 95% bootstrap theo prompt (10^4 lần, seed 0). Thời gian gồm cả prefill.

6.6 Kết quả

6.6.1 Tính đúng đắn

6.6.2 Toán tử nào làm vỡ tính exact

Để biết sai lệch của kiểm tra theo khối đến từ đâu, chúng tôi chạy đường *chuẩn* (không vá) của HF với bitsandbytes 0.50.0 trên các khối ba token teacher-forced, đặt ở trạng thái tuần tự chính xác (20 prompt GSM8K dev, 16 khối mỗi prompt, 960 hàng). Với mỗi toán tử ta ghi đầu vào thật của nó khi chạy khối, rồi gọi lại *cùng toán tử trên cùng đầu vào* từng hàng một và so sánh từng bit (Bảng 6.3). Chỉ attention và LM head lệch đáng kể; các phép chiếu NF4 chuẩn khớp từng hàng ở mọi phần tử đã kiểm, và RMSNorm gần như khớp. Hệ quả cuối-cuối: $960/960$ hàng logits không bằng từng bit, $4/960$ hàng (0.42%) đổi argmax (độ chênh top-1/top-2 của

Bảng 6.3: So sánh từng bit giữa toán tử gọi ba hàng và gọi từng hàng trên cùng đầu vào (đường chuẩn, 960 hàng).

Toán tử	Số phần tử	Lệch từng bit	Tỷ lệ
Phép chiếu NF4 (7 loại)	990 904 320	0	0
RMSNorm trong lớp	141 557 760	87	6.1×10^{-7}
RMSNorm cuối	1 966 080	0	0
LM head	145 858 560	281 534	1.9×10^{-3}
Attention (SDPA)	70 778 880	13 196 548	0.186

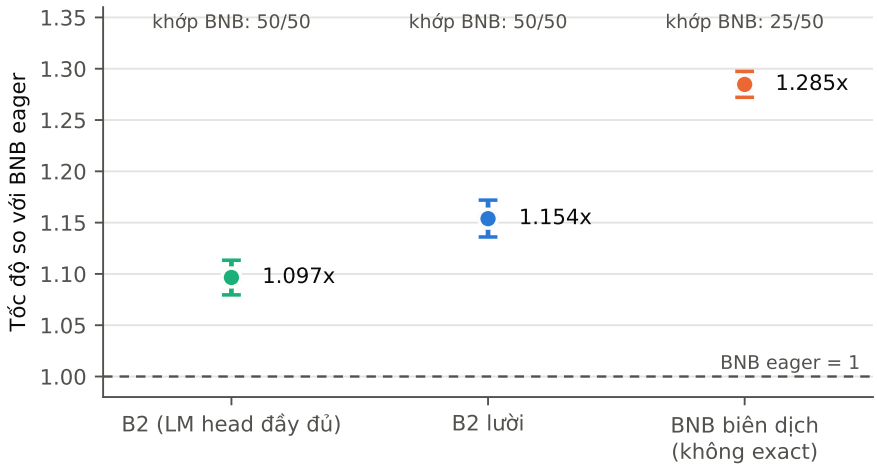
chúng ≤ 0.125, một hàng là hòa tuyệt đối); trung vị của độ lệch logits lớn nhất là 0.22 và giá trị lớn nhất là 1.36. Vì vậy trong ngăn xếp này tính exact bị phá bởi attention và LM head chứ không phải phép chiếu 4-bit, và B2 xử lý chính các toán tử đó bằng lời gọi từng hàng. Kết quả chỉ cho một phiên bản bitsandbytes, một GPU và $M{=}3$: bitsandbytes 0.50.0 chọn kernel NF4 cho M nhỏ theo ngưỡng riêng, và phiên bản khác có thể khác.

6.6.3 Tốc độ và so sánh với B1

Trên held-out 300–349 (phiên bản B2 với LM head đầy đủ), B2 đạt 1.117 [1.100, 1.135]$\times$ so với BNB eager, khớp $50/50$ prompt; B1 (kernel Triton ba hàng, lịch hình dạng cố định) đạt 1.108 [1.087, 1.130]$\times$ nhưng chỉ khớp BNB gốc ở $24/50$ prompt. Tỷ số B2/B1 là 1.008 [0.995, 1.022]: hai bên không phân biệt được về tốc độ, khác biệt nằm ở tính exact. Trên held-out 400–449 với B2 lười (Hình 9): B2 lười đạt 1.154 [1.136, 1.172]$\times$ so với 1.097 [1.080, 1.113]$\times$ của B2 LM head đầy đủ (lười/đầy đủ $= 1.052[1.044,\ 1.060]$), không prompt nào dưới BNB. Trên dev, B2 lười đạt 1.166 [1.145, 1.188]$\times$ (GSM8K) và 1.249 [1.208, 1.296]$\times$ (HumanEval), mỗi cohort 30 prompt.

6.6.4 So sánh với baseline biên dịch

Một vòng decode BNB với `StaticCache` và `torch.compile` (mode `reduce-overhead`) nhanh hơn BNB eager 1.285 [1.272, 1.297]$\times$ trên held-out 400–449, đo cạnh nhau cùng lượt chạy. Tuy nhiên token của nó chỉ khớp eager ở $25/50$ prompt, tức không phải tham chiếu bitwise. B2 lười đạt $0.907\times$ [0.893, 0.920] tốc độ của nó và chậm hơn ở $49/50$ prompt (trên dev: $0.82\times$, 12 prompt). Vì vậy luận điểm đúng của B2 là "exact so với BNB eager, nhanh hơn eager khoảng 1.1–$1.25\times$", không phải "nhanh hơn baseline mạnh". Chưa thử kết hợp compile với giải mã suy đoán; nó sẽ đổi số học và đòi hỏi định nghĩa lại tham chiếu.



Hình 9: Tốc độ so với BNB eager trên held-out GSM8K 400–449 (50 prompt; điểm là trung bình nhân, thanh là khoảng tin cậy 95%). Dòng trên cùng ghi số prompt có token khớp BNB eager. BNB biên dịch nhanh hơn nhưng chỉ khớp $25/50$.

6.6.5 Ngữ cảnh dài và các mô hình khác

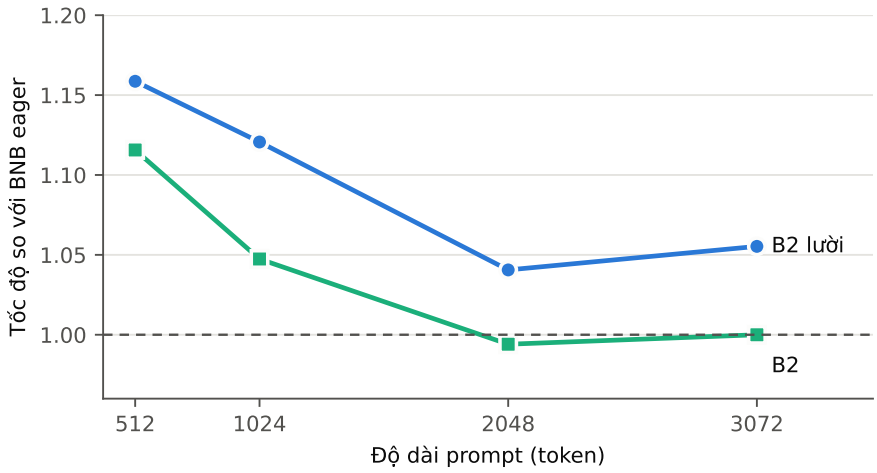
Token và KV khớp ở mọi độ dài tới 3071 token, nhưng tốc độ giảm (Hình 10): B2 lười còn 1.04–$1.06\times$ ở 2048–3072 token. Chi phí một forward gần như không đổi theo độ dài (wall: BNB $M{=}1$ $23.2 \rightarrow 23.5$ ms), nên nguyên nhân đã đo là acceptance thấp trên tác vụ tóm tắt (0.24–0.29 ở 3072 so với 0.32–0.64 ở 512) và prefill chung (1.1 s ở 3072 token) làm loãng tỷ lệ; chưa có prompt dài mà acceptance cao để tách hoàn toàn hai yếu tố. Trên Qwen2.5-0.5B và 1.5B (10 prompt dev) B2 lười đạt $1.243\times$ và $1.163\times$, token và KV khớp $10/10$.

6.6.6 Bộ nhớ

Ở cùng dung lượng cache tĩnh, đỉnh bộ nhớ cấp phát của B2 chỉ lớn hơn BNB eager 0.07 MiB (2007.90 so với 2007.97 MiB ở prompt 75 token; 2994.61 so với 2994.68 MiB ở 3071 token); đỉnh dự trữ và bộ nhớ thiết bị dùng giống hệt nhau, và không có mô hình draft. Mỗi phương pháp chạy trong một tiến trình riêng, đo ở lượt thứ hai.

6.6.7 Phân tích chi phí

Một forward khối $M{=}3$ tốn 37.1 ms, gấp 1.65 lần một bước $M{=}1$ (22.5 ms), nên block chỉ có lợi khi sinh trên 1.65 token mỗi chu kỳ (acceptance khoảng 33% với hai đề xuất); trung bình 1.86 token mỗi chu kỳ khối và 73% chu kỳ phải quay về bước gốc, khớp với mức tăng tốc khoảng 12–15%. LM head bf16 tốn khoảng 3.3 ms mỗi hàng và là phần lớn của chênh lệch GPU, nên chiếu lười có ích. Cả hai loại forward bị giới hạn bởi CPU khởi chạy kernel



Hình 10: Tốc độ so với BNB eager theo độ dài prompt (3 prompt dev mỗi độ dài, tóm tắt nhiều bài CNN/DM ghép lại).

(84% thời gian một bước $M{=}1$ là enqueue CPU), điều này cũng giải thích vì sao baseline biên dịch (giảm overhead CPU) nhanh hơn.

6.6.8 Thăm dò: chụp CUDA graph (ước lượng, chưa cài đặt)

Chúng tôi chụp thô một bước decode $M{=}1$ của mô hình bitsandbytes eager với `StaticCache` bằng CUDA graph (không `torch.compile`, cùng các kernel): chụp được, logits bằng từng bit ở $64/64$ bước, và chi phí giảm từ 22.5 xuống 17.3 ms mỗi bước ($1.30\times$). Khối chuẩn ba hàng (không exact) tốn 27.4 ms ở chế độ eager và 23.9 ms trong graph. Ghép với mô phỏng đã hiệu chỉnh, một vòng B2 chạy phần thân decoder trong graph (các phép từng hàng nằm trong graph nên không còn tốn CPU) với LM head lười chạy ngoài graph cho tốc độ ước lượng 1.36–$1.48\times$ so với eager, nhưng chỉ 1.04–$1.14\times$ so với một baseline graph exact. Đây là dự đoán từ mô hình chi phí (chi phí decoder ba hàng trong graph 21.7–24.0 ms ước từ số đo kernel), chưa được cài đặt, nên không phải kết quả đã đo của B2.

6.7 Các kết quả bất lợi và hướng không hiệu quả

- **Chọn n-gram.** Quét độ dài n-gram 2–5 chênh dưới 2% và nằm trong nhiễu đo. Bảy chính sách chọn draft (lọc đề xuất yếu, bỏ phiếu giữa các lần khớp, ngưỡng lặp) đều khớp BNB; lọc đề xuất yếu không nhanh hơn dù giảm gần một nửa số khối, và bỏ phiếu so với chính sách gốc cho tỷ số 1.002 [0.998, 1.007] trên 80 prompt dev chưa dùng để chọn. Giữ chính sách gốc.

- **Giảm chi phí CPU.** Gọi thẳng kernel Triton đã biên dịch cho kết quả bằng nhau từng bit nhưng không nhanh hơn (7.22 so với 7.59 ms cho 252 phép chiếu); các điểm nóng còn lại là chuỗi phép PyTorch nhỏ ở RMSNorm và attention từng hàng, không thể giảm mà không đổi tập kernel.
- **Khối lớn hơn.** Mô phỏng (chạy PLD trên chuỗi tham chiếu đã ghi, chi phí mỗi hàng hiệu chỉnh khớp $K{=}2$ đo được) dự đoán $K{=}3, 4$ thay đổi từ -1% đến $+2.6\%$ so với $K{=}2$ (tới $+4.1\%$ nếu chi phí mỗi hàng giảm một nửa); không đáng viết kernel mới. Đây là dự đoán từ mô hình tuyến tính, chưa kiểm bằng thực nghiệm.
- **Quay về bước gốc.** Quay về $M{=}1$ khi không có đề xuất đưa tốc độ từ 0.91–$0.97\times$ lên 1.06–$1.15\times$ trên các cohort dev nhỏ; đây là thay đổi đóng góp lớn nhất.

6.8 Giới hạn và mối đe dọa tính hợp lệ

- **Phạm vi.** Một GPU và bộ công cụ, chủ yếu một họ mô hình (Qwen2), một backend lượng tử hóa (bitsandbytes), batch 1, greedy, $k{=}2$. Kết quả không áp dụng được cho lấy mẫu, batch lớn hay các lược đồ lượng tử hóa khác.
- **Chứng minh.** Tính exact dựa trên bằng chứng hữu hạn. Còn mở: bằng kernel bitsandbytes trên mọi đầu vào, độc lập của dispatch PyTorch với số hàng, độ ổn định qua phiên bản Triton/ptxas/CUDA và GPU, kiểm ở mức PTX.
- **Chọn cấu hình.** Cấu hình được chọn trên dev; hai cohort held-out không dùng để chọn, nhưng cohort 400–449 được dùng để đánh giá riêng biến thể B2 lười đã cố định trước đó, và cohort 300–349 chỉ đánh giá biến thể LM head đầy đủ.
- **Khoảng tin cậy.** Chỉ phản ánh biến thiên giữa các prompt đã chọn, không phải toàn bộ workload. Tốc độ tuyệt đối phụ thuộc cohort ($1.23\times$ ở HumanEval 0–19 nhưng $1.31\times$ ở 20–49).
- **Baseline.** Tốc độ chỉ so với BNB eager; baseline biên dịch nhanh hơn nhưng không exact.

6.9 Tóm tắt chương

Với Qwen2.5-3B NF4 trên một GPU tiêu dùng, B2 tái tạo đúng từng token và KV của greedy bitsandbytes, không dùng thêm bộ nhớ đáng kể và nhanh hơn eager khoảng 12–15% trên hai cohort held-out; mức tăng khiêm tốn và bị giới hạn bởi chi phí khởi chạy CPU và acceptance của PLD. Giá trị chính nằm ở tính exact và quy trình bằng chứng chứ không ở tốc độ. Hướng tiếp theo gồm kiểm ở mức PTX và trên GPU/phiên bản Triton khác, mở rộng sang backend lượng tử hóa và họ mô hình khác, và chứng minh hợp đồng toán tử.

Mã nguồn và bằng chứng. Mã, bằng chứng thô, manifest ghim SHA256, kịch bản tái lập, lập luận theo toán tử và ghi chép trạng thái nghiên cứu đều nằm trong kho mã của dự án (thư mục `experiments` và các tệp ở thư mục gốc); số liệu trong chương được lấy trực tiếp từ các tệp JSON bằng chứng đó.

Kernel 4-bit bất biến theo số hàng cho giải mã suy đoán exact

Chương 6 xây dựng vòng giải mã exact B2 bằng cách gọi từng hàng các toán tử nhạy với hình dạng. Cách đó giữ đúng chuỗi của bitsandbytes gốc nhưng bỏ phần lớn lợi ích của việc kiểm tra theo khối: trọng số được đọc lại cho mỗi hàng. Chương này đi theo hướng ngược lại, trả lời RQ6: *có thể thiết kế các kernel 4-bit mà kết quả của từng hàng không phụ thuộc vào số hàng M trong khối, sao cho kiểm tra theo khối vừa exact vừa thực sự đọc trọng số một lần?* Cái giá phải trả là tham chiếu exact không còn là bitsandbytes gốc mà là chính bộ kernel mới chạy ở $M{=}1$; mục 7.2 đo sự khác biệt này.

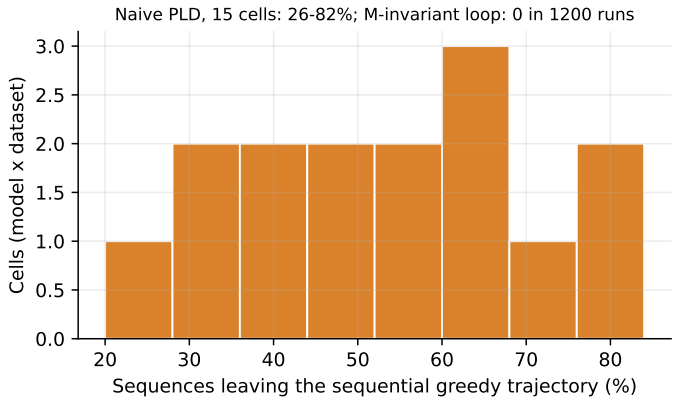
7.1 Vấn đề: kernel đổi chiến lược theo M

Trên một GPU, thư viện chọn kích thước tile, hệ số chia chiều rút gọn (split-K) hoặc cả kernel khác nhau tùy số hàng. Hệ quả là cùng một hàng kích hoạt cho kết quả khác từng bit khi nằm trong khối $M{=}1$ hay $M{=}4$. Bảng 7.1 tóm tắt phép kiểm toán theo toán tử trên bitsandbytes 0.50.0 (RTX 4050, sm_89): phép chiếu NF4 ổn định đến $M{=}3$ rồi lệch khoảng 13–15% phần tử từ $M{\geq}4$, khớp với việc bitsandbytes đổi kernel ở $M{=}4$; attention (SDPA) lệch 3.6–19.2% phần tử ở mọi M, LM head lệch 0–0.19%, RMSNorm lệch ở một số mô hình. Ngưỡng $M{=}4$ chỉ đo trên phiên bản và GPU này; bản 0.50.1/0.50.2 đổi cách chọn kernel nên không suy rộng được.

Hệ quả cuối-cuối: PLD ngây thơ (kiểm tra theo khối, kernel gốc) rời khỏi quỹ đạo greedy tuần tự ở 26–82% số chuỗi trên 15 ô (5 mô hình $\times$ 3 tập, 50 prompt mỗi ô; tới 92% trên một tập CNN/DM độc lập), còn một mô hình nháp thật (Qwen2.5-0.5B nháp cho 3B, $K{=}2, 4, 6$) lệch 34–38% trên GSM8K và 64–66% trên CNN/DM. Chạy từng hàng đưa cả hai về 0 lệch (Hình 11). Quan sát định tính rằng kiểm tra theo khối trên backend lượng tử hóa làm đổi logit và đổi argmax đã được báo cáo trên Metal [19]; đóng góp ở đây là đo theo từng toán tử trên CUDA và phần sửa.

Bảng 7.1: Độ lệch từng bit so với chạy từng hàng, bitsandbytes 0.50.0 NF4, GSM8K, 960 hàng mỗi ô. "Proj": tỷ lệ phần tử sai của các phép chiếu NF4; RMSNorm là số phần tử sai.

Mô hình	M	Proj	RMSNorm	LM head	SDPA
Qwen2.5-3B	2	0%	60	0.193%	18.6%
Qwen2.5-3B	3	0%	87	0.193%	18.6%
Qwen2.5-3B	4	12.96%	244	0.193%	18.6%
Qwen2.5-3B	8	12.96%	199	0.193%	18.6%
Qwen2.5-1.5B	4	13.85%	92	0.109%	18.3%
Qwen2.5-0.5B	4	14.27%	0	0.000%	3.6%
OLMo-2-1B	4	15.42%	189	0.048%	17.4%
Phi-3.5-mini	4	14.79%	459	0.083%	19.2%



Hình 11: Phân bố tỷ lệ chuỗi lệch quỹ đạo greedy tuần tự của PLD ngây thơ trên 15 ô (mô hình $\times$ tập dữ liệu).

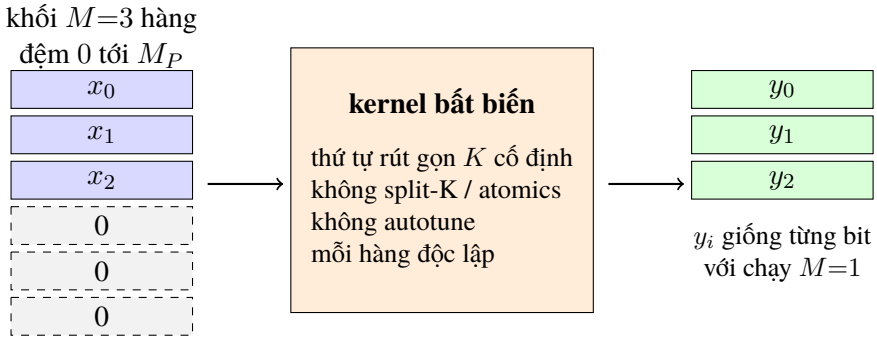
7.2 Định nghĩa exact và tham chiếu

Gọi $\mathcal{K}$ là một bộ kernel cố định và $R_{\mathcal{K}}$ là giải mã greedy tuần tự dùng $\mathcal{K}$ với $M{=}1$ ở mọi bước. Một bộ giải mã suy đoán là *exact theo* $\mathcal{K}$ nếu với mọi prompt nó trả về đúng các token của $R_{\mathcal{K}}$ và để lại đúng các byte KV cache của $R_{\mathcal{K}}$. Ta *không* đòi bằng greedy bitsandbytes gốc: đó là một bộ kernel khác.

Hai bộ kernel hợp lệ khác thứ tự cộng thì tách nhau. Phép đo trên Qwen2.5-3B (Bảng 7.2) cho thấy chuỗi từ tham chiếu bất biến giống hệt chuỗi bitsandbytes gốc ở chỉ 30% ($n{=}50$) và 44% ($n{=}150$) số prompt GSM8K, cùng cỡ với mức lệch của PLD ngây thơ. Độ chính xác tác vụ không bị phát hiện giảm: GSM8K 72% so với 66% ($n{=}50$, 5 so với 2 cặp bất đồng, trong nhiễu). Đây là kiểm tra không phát hiện sai khác, không phải chứng minh tương đương. Trên HumanEval ($n{=}100$, prompt thô, tối đa 384 token) pass@1 là $61/100$ ở cả hai chế độ, với 3 và 3 bài bất đồng theo hai chiều; chuỗi token giống hệt ở $72/100$ prompt (mức giống cao hơn GSM8K vì đầu ra ngắn hơn). Con số 61% thấp hơn mức công bố cho mô hình vì ta dùng

Bảng 7.2: Tham chiếu bất biến so với greedy bitsandbytes gốc, Qwen2.5-3B NF4.

Phép đo	n	Gốc	Bất biến
Chuỗi giống hệt (GSM8K)	50	15 (30%)	
Chuỗi giống hệt (GSM8K, ext150)	150	66 (44%)	
Độ chính xác GSM8K	50	33 (66%)	36 (72%)
Chuỗi giống hệt (HumanEval)	100	72 (72%)	
pass@1 HumanEval	100	61 (61%)	61 (61%)



Hình 12: Nguyên tắc thiết kế: khối M hàng được đệm 0 tới kích thước cố định M_P và đi qua cùng một dãy lệnh; kết quả mỗi hàng không phụ thuộc M.

prompt thô và cắt hoàn thành bằng quy tắc đơn giản, nên chỉ có ý nghĩa để so hai chế độ với nhau.

7.3 Thiết kế kernel bất biến

Nguyên tắc. Phần tử đầu ra (m, n) được tính bởi cùng một dãy lệnh bất kể M và các hàng khác. Có bốn quy tắc: (i) đệm số 0 khối hàng về kích thước cố định M_P; (ii) thứ tự rút gọn theo K cố định; (iii) không split-K, không atomics, không tự điều chỉnh (autotune); (iv) không có phụ thuộc dữ liệu giữa các hàng. Bất biến được kiểm bằng thực nghiệm (hàng chục nghìn hàng mỗi kernel); chương này không chứng minh hình thức. Hình 12 minh họa nguyên tắc.

LM head. Kernel Triton dùng `tl.dot` trên khối 16 hàng (đệm 0), fp16/bf16 $\rightarrow$ fp32, K chia khối 128 theo thứ tự cố định. Kết quả bằng từng bit với GEMM cuBLAS khi $M \geq 2$ nhưng không bằng đường gemv của cuBLAS ở M=1 (1.2% phần tử khác); đây là lý do tham chiếu không trùng bitsandbytes gốc. Chi phí 3.42 ms so với 3.34 ms của cuBLAS ở M=1 (1.02×) và gần như không đổi theo M=1..8; chạy từng hàng ba lần tốn 10.8 ms.

Attention. Kernel Triton split-KV với các đoạn khóa tuyệt đối (256 khóa, tile 64): mỗi đoạn tính softmax trực tuyến riêng, các kết quả riêng được gộp theo thứ tự đoạn tăng dần. Đoạn bị

Thuật toán 3 Kernel GEMM NF4 bất biến theo M (một warp).

Đầu vào: R hàng trọng số NF4, $M \le M_P$ hàng kích hoạt (đệm 0 tới M_P)
Đầu ra: $Y[m, r]$ với $m < M$, độ lớn không phụ thuộc M

for $k_0 = 0, 1024, 2048, \ldots$ **do** ▷ thứ tự cố định
    mỗi lane nạp 32 nibble của mỗi hàng trọng số, giải lượng tử một lần
    for $m = 0..M_P - 1$, $r = 0..R - 1$ **do**
        $acc[m, r] \leftarrow \text{fmaf}(x[m, k], w[r, k], acc[m, r])$ theo k tăng dần
    end for
end for
rút gọn $acc[m, r]$ qua 32 lane bằng xor-butterfly cố định
cộng bias (fp32) và làm tròn về dtype đầu ra

Bảng 7.3: Kiểm chứng bất biến theo M (số hàng lệch / tổng số hàng kiểm).

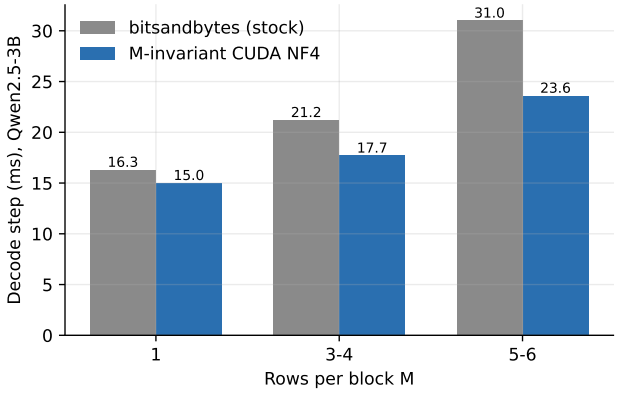
Kernel	Hàng kiểm	Lệch
LM head (5 hình dạng)	27 000	0
Attention (Qwen 3B, 1.5B, 0.5B; ngữ cảnh 128–3000)	3×6300	0
Attention MHA $32/32$	2 100	0
GEMM NF4 (mọi biến thể M_P, R)	7 000	0

che hoàn toàn là phép không (no-op) chính xác, nên kết quả một hàng không phụ thuộc vào số hàng hay vị trí sau nó. Ở chế độ CUDA graph, độ dài tiền tố là một số `int32` trên thiết bị. GQA dùng K/V gọn. Ở $M{=}3$ nhanh hơn SDPA có mặt nạ 5–25 lần.

RMSNorm. Mỗi chương trình xử lý đúng một hàng.

GEMM NF4. Kernel CUDA C++ dùng bố cục bitsandbytes (nibble cao trước, blocksize 64, absmax lượng tử hóa kép). Một warp xử lý R hàng đầu ra và M_P hàng kích hoạt; trọng số được đọc và giải lượng tử một lần cho mọi hàng, đây là nơi có lợi ích so với chạy từng hàng. Mỗi lane xử lý 32 phần tử K mỗi bước 1024 phần tử, cộng dồn `fmaf` theo thứ tự cố định rồi rút gọn xor-butterfly. Các biến thể $M_P \in \{1, 2, 4, 8\}$ và $R \in \{1, 2, 4, 8\}$ cho cùng từng bit trên mỗi hàng (0 lệch trên 7000 hàng). Hai phiên bản Triton đầu chậm hơn bitsandbytes 4–20 lần nên bị bỏ.

Một lỗi mà kiểm tra "exact" không bắt được. Bản đầu của kernel NF4 đọc bias fp16 như bf16 và cho đầu ra vô nghĩa trong mô hình thật, nhưng vẫn "exact" 100% vì mọi cách chạy đều sinh cùng một chuỗi lặp vô nghĩa. Lỗi chỉ lộ khi đọc văn bản sinh ra; đã sửa (bias chuyển sang fp32) và so từng lớp với ma trận giải lượng tử fp32 trên kích hoạt thật. Bài học: kiểm tra exact phải đi kèm kiểm tra văn bản hoặc độ chính xác tác vụ. Mọi số liệu tốc độ trong chương đều sinh sau bản sửa.



Hình 13: Thời gian một bước giải mã trong CUDA graph, Qwen2.5-3B NF4, theo số hàng của khối.

Thuật toán 4 Vòng PLD exact với kernel bất biến.

Đầu vào: prompt, tham chiếu $R_{\mathcal{K}}$, $K{=}2$

prefill bằng kernel bất biến; $n \leftarrow$ độ dài tiền tố
while chưa EOS và chưa đủ token **do**
  $d \leftarrow$ tra n-gram (tối đa K token) ▷ có thể rỗng
  chạy khối $1{+}|d|$ hàng bằng đồ thị CUDA ($M \in \{1, 2, 3\}$)
  $j \leftarrow$ số token nháp khớp argmax; commit $j{+}1$ token
  `cumulative_length.fill_(n + j + 1)`; $n \leftarrow n + j + 1$ ▷ quay lui KV
end while

7.4 Chi phí một bước giải mã

Thời gian một bước giải mã đầy đủ trong CUDA graph (Qwen2.5-3B): $M{=}1$ 15.0 ms (bitsandbytes 16.3), $M{=}3$–4 17.7 ms (bitsandbytes 21.2 đến 31.5), $M{=}5$–6 23.6 ms (bitsandbytes 31) (Hình 13). Kernel GEMM chiếm 9.2 ms ở $M{=}1$, chậm hơn `gemm_4bit_simt` của bitsandbytes (8.9 ms) khoảng 3% nhưng nhanh rõ ở $M{\geq}3$. Nghĩa là bước kiểm tra $M{=}3$ chỉ đắt hơn bước $M{=}1$ khoảng 18%, đúng điều kiện để giải mã suy đoán có lợi.

7.5 Vòng giải mã exact trên CUDA graph

PLD đề xuất tối đa $K{=}2$ token (n-gram 3); khối $M \leq 3$ hàng được kiểm tra bằng các kernel bất biến trong một lần phát lại CUDA graph; token được chấp nhận được commit, hàng bị loại được quay lui bằng cách đặt lại độ dài của static cache (`cumulative_length.fill_(n)`); tensor tiền tố của attention được cập nhật trước mỗi lần phát lại. Vì mọi kernel bất biến theo M, byte KV đã commit bằng của giải mã tuần tự, điều ta kiểm trực tiếp.

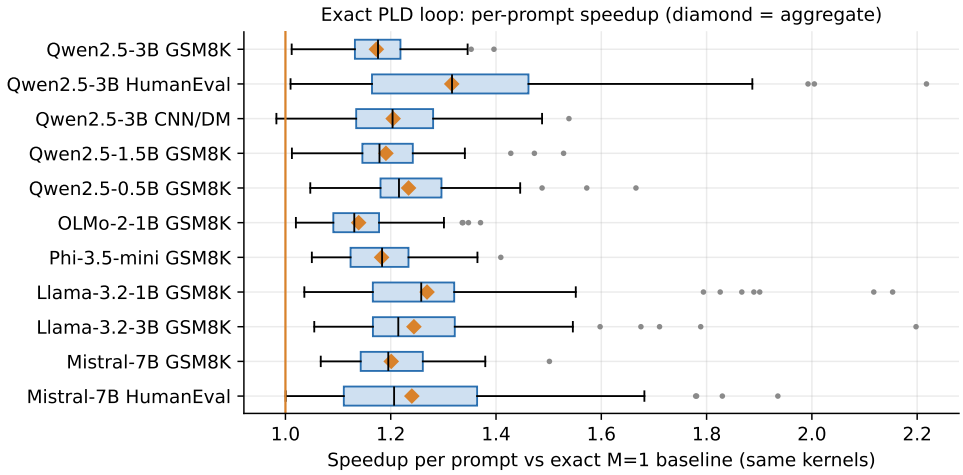
Bảng 7.4: Vòng PLD exact so với baseline $M{=}1$ cùng kernel. Tăng tốc theo prompt là trung vị [min, max]. Cận trên Wilson 95% của tỷ lệ lệch: 1.9% ($n{=}200$), 3.7% ($n{=}100$).

Ô	Prompt	Exact	Base	PLD	Tăng	Theo prompt
Qwen2.5-3B GSM8K	200	200/200	61.2	71.8	1.17×	1.18 [1.01, 1.40]
Qwen2.5-3B HumanEval	100	100/100	58.9	77.6	1.32×	1.32 [1.01, 2.22]
Qwen2.5-3B CNN/DM	100	100/100	58.7	70.7	1.21×	1.21 [0.98, 1.54]
Qwen2.5-1.5B GSM8K	100	100/100	97.1	115.6	1.19×	1.18 [1.01, 1.53]
Qwen2.5-0.5B GSM8K	100	100/100	196.0	241.9	1.23×	1.22 [1.05, 1.67]
OLMo-2-1B GSM8K	100	100/100	136.2	155.1	1.14×	1.13 [1.02, 1.37]
Phi-3.5-mini GSM8K	100	100/100	72.7	85.9	1.18×	1.18 [1.05, 1.41]
Llama-3.2-1B GSM8K	100	100/100	135.1	171.5	1.27×	1.26 [1.04, 2.15]
Llama-3.2-3B GSM8K	100	100/100	62.9	78.2	1.24×	1.21 [1.05, 2.20]
Mistral-7B GSM8K	100	100/100	38.1	45.8	1.20×	1.20 [1.07, 1.50]
Mistral-7B HumanEval	100	100/100	35.5	44.1	1.24×	1.21 [1.00, 1.94]

7.6 Kết quả đầu-cuối

Thiết lập. RTX 4050 Laptop 6 GB, Triton 3.6.0, nvcc 12.8, bitsandbytes 0.50.0, greedy, batch 1, 128 token mới, $K{=}2$, n-gram 3. Baseline là giải mã $M{=}1$ exact dùng cùng kernel bất biến trong CUDA graph, cùng phiên và cùng prompt. "Exact" là token sinh ra *và* KV đã commit bằng từng bit với $R_{\mathcal{K}}$.

Tổng cộng 1200 cặp (prompt, mô hình) trên 8 mô hình (năm họ Qwen2.5, OLMo-2, Phi-3.5, Llama-3.2 và Mistral; GQA và MHA; $head_dim$ 64, 96 và 128): 0 lệch về token và KV; các lần chạy lặp (cùng phiên so sánh, kiểm tra EOS, kiểm tra phiên bản) cũng đều exact. Tăng tốc 1.14–1.32×; chỉ một prompt (CNN/DM) chậm hơn baseline (0.98×) (Hình 14). Phi-3.5 cần hai chỉnh sửa hạ tầng chứ không đổi kernel: kernel attention được mở rộng cho $head_dim$ 96 (đệm lên 128 trong kernel bằng mặt nạ, 0 lệch trên 700 hàng, bốn kernel cũ vẫn 0 lệch), RoPE LongRoPE được cố định ở chế độ hệ số ngắn (đúng với ngữ cảnh dưới 4096 token, vì bản gốc chọn hệ số bằng một phép đồng bộ dữ liệu làm hỏng CUDA graph), và cache dùng lớp tĩnh thông thường thay cho lớp cửa sổ trượt (cửa sổ 262144 lớn hơn ngữ cảnh, lớp cửa sổ trượt giữ độ dài bằng số nguyên Python nên không quay lui được). Lần chạy đầu của Phi-3.5 trước khi sửa lớp cache cho tốc độ cao bất thường (1.74×) nhưng không exact; cờ exact đã phát hiện được lỗi này. Hai mô hình Llama-3.2 và Mistral-7B-Instruct-v0.3 (bản sao không cần token của unsloth; Mistral là mô hình 7B đầu tiên, tải sẵn dạng NF4 và vừa GPU 6 GB với cache ngắn) chạy không cần sửa gì. Trên Mistral-7B, một bước giải mã $M{=}1$ tốn khoảng 26–28 ms, gần gấp đôi mô hình 3B, nhưng mức tăng của PLD (1.20–1.24×) tương đương các mô hình nhỏ; ở HumanEval, tăng tốc chỉ trên 55 prompt không có EOS là 1.28×. Baseline $M{=}1$ bất biến còn nhanh hơn baseline graph kernel gốc (62.4 so với 57.8 tok/s trên GSM8K).



Hình 14: Phân bố tăng tốc theo prompt của vòng PLD exact so với baseline $M{=}1$ cùng kernel.

So với vòng exact cũ chạy từng hàng. Cùng prompt, chạy liền nhau (vòng cũ, vòng mới, vòng cũ để đo trôi): với 100 prompt mỗi tập, vòng mới đạt 79.0 tok/s trên GSM8K so với 62.7 và 61.2 của vòng cũ ($1.26\times$ và $1.29\times$), 76.5 so với 56.4 và 57.0 trên CNN/DM ($1.36\times$ và $1.34\times$), 81.9 so với 58.9 và 59.4 trên HumanEval ($1.39\times$ và $1.38\times$); tất cả các lượt đều exact. Hai lần chạy vòng cũ lệch nhau nhiều nhất 2.4%, nên nhiễu giữa các phiên nhỏ hơn mức chênh. So với baseline $M{=}1$ bất biến cùng phiên, mức tăng trong các phiên này là $1.18\times$, $1.20\times$ và $1.31\times$ (nhanh hơn một chút so với Bảng 7.4 do phiên chạy khác nhau).

Ablation. PLD với $K{=}4$ chậm hơn $K{=}2$ ($1.145\times$ so với $1.206\times$, 10 prompt). Tắt làm tròn trọng số hay thay tra bảng bằng phép tính chỉ đổi thời gian GEMM dưới 5%; ở $M{=}4$ kernel gần giới hạn số phép FMA, ở $M{=}1$ còn cách giới hạn băng thông khoảng 20%. Ước tính tối đa 10% thời gian bước còn thu hồi được, tăng đều cho cả baseline lẫn vòng PLD nên gần như không đổi tỷ số tăng tốc.

Kiểm tra văn bản sau EOS. Vòng sinh đúng 128 token và không dừng ở EOS, nên với prompt có EOS sớm phần sau là văn bản lặp, thứ mà PLD đoán rất dễ và có thể làm tăng tốc cao hơn thực tế. Ta chạy lại sáu ô (100 prompt mỗi ô) ghi vị trí EOS đầu tiên của chuỗi tham chiếu: tỷ lệ prompt có EOS trong 128 token là 0% (Qwen-3B GSM8K), 26% (HumanEval), 8% (CNN/DM), 9% (Llama-3B), 17% (Llama-1B) và 15% (Phi-3.5). Tăng tốc chỉ tính trên các prompt không có EOS lần lượt là 1.18, 1.32, 1.21, 1.23, 1.29 và $1.19\times$, so với 1.18, 1.32, 1.20, 1.24, 1.27 và $1.18\times$ khi tính mọi prompt (lệch dưới 2%); cả 600 lượt chạy này đều exact. Văn bản sau EOS vì vậy không làm tăng số đo.

Bảng 7.5: Tỷ lệ phần tử NF4 khác từng bit giữa khối M hàng và chạy từng hàng (stock bitsandbytes, hình dạng 11008×2048, dữ liệu ngẫu nhiên).

Phiên bản	$M{=}2$	$M{=}3$	$M{=}4$	$M{=}8$	$M{=}16$
0.49.2	60.5%	60.2%	60.2%	60.3%	60.4%
0.50.0	0%	0%	60.2%	60.3%	60.4%
0.50.1	0%	0%	60.2%	60.3%	60.4%

7.7 Độ phụ thuộc vào phiên bản bitsandbytes

Kernel GEMM NF4 của ta chỉ dùng bố cục lượng tử hóa của bitsandbytes, không gọi kernel nhân của nó, nên bộ kernel không phụ thuộc cách bitsandbytes chọn kernel. Ngược lại, hành vi của đường gốc mà ta thay thế phụ thuộc phiên bản. Ta đo tỷ lệ phần tử khác từng bit giữa phép chiếu NF4 chạy cả khối M hàng và chạy từng hàng, trên dữ liệu ngẫu nhiên với ba hình dạng của Qwen2.5-3B (Bảng 7.5): bitsandbytes 0.50.0 và 0.50.1 cho kết quả giống hệt nhau (ổn định đến $M{=}3$ và lệch $\approx 60\%$ phần tử từ $M{\geq}4$), còn 0.49.2 (trước PR #1949 thêm kernel GEMM 4-bit hợp nhất) lệch $\approx 60\%$ ngay từ $M{=}2$. Nói cách khác ngưỡng $M{=}4$ chỉ là tính chất của 0.50.x; trên 0.49.2 ngay cả khối hai hàng cũng không an toàn. Bộ kernel bất biến vượt cả bốn kiểm tra đơn vị và vòng exact trên 20 prompt GSM8K với cả 0.49.2 và 0.50.1 ($20/20$ exact, tăng $1.19\times$ và $1.20\times$ so với baseline $M{=}1$ bất biến). Các tỷ lệ $\approx 60\%$ lớn hơn 13–15% ở Bảng 7.1 vì dữ liệu ngẫu nhiên fp16 khác kích hoạt thật; chỉ so sánh giữa các phiên bản mới có ý nghĩa.

7.8 Kết quả âm

- **Mô hình nháp thần kinh.** Qwen2.5-0.5B bf16 nháp cho 3B NF4 ($K{=}2$) exact nhưng chỉ $0.66\times$ baseline: một bước nháp tốn khoảng 10 ms so với 15 ms của một bước đích. Tính từ số đo, ngay cả nháp nén NF4 cũng chỉ khoảng $0.87\times$, và nháp miễn phí bị chặn ở khoảng $1.36\times$. Với đích 4-bit 3B đã rất rẻ trên GPU này, PLD là nguồn nháp phù hợp. Với đích $\geq$ 7B tình hình có thể khác nhưng chưa đo được.
- **Tối ưu kernel** còn tối đa khoảng 10% (mục trước).
- Tiêu chí ban đầu "$\geq 1.15\times$ với mô hình nháp thật" *không đạt*; chỉ PLD đạt tăng tốc.

7.9 Giới hạn và mối đe dọa tính hợp lệ

(i) Một GPU (RTX 4050 6 GB), mô hình tới 7B (chỉ một mô hình 7B, với cache ngắn), batch 1, chỉ greedy; chưa chạy mô hình $\geq$ 7B hay GPU khác. (ii) Bất biến dựa vào cấu trúc kernel

và thực nghiệm, không phải chứng minh, và phụ thuộc phiên bản trình biên dịch (Triton 3.6.0, nvcc 12.8); các kiểm tra bất biến và vòng exact (20 prompt) cũng đạt với bitsandbytes 0.49.2 và 0.50.1 (xem mục 7.7), nhưng chỉ trên một GPU. (iii) Chưa thử họ kiến trúc nào ngoài Qwen2, OLMo-2, Phi-3 và Llama-3.2 (chưa có Gemma và mô hình MoE). (iv) Tăng tốc phụ thuộc mức PLD tìm được bản nháp: cao nhất ở HumanEval, thấp nhất ở OLMo-2. (v) So sánh độ chính xác với tham chiếu gốc còn nhỏ (n=50 trên GSM8K). (vi) Phi-3.5 chỉ được hỗ trợ trong chế độ ngữ cảnh ≤ 4096 token (hệ số LongRoPE ngắn). (vii) Mọi prompt dùng văn bản thô (không dùng chat template). (viii) So sánh với LLM-42 và các công trình đồng thời mới ở mức đọc tài liệu.

7.10 Tóm tắt chương

Bộ kernel bất biến theo M (LM head, attention, RMSNorm, GEMM NF4) cho phép kiểm tra theo khối exact theo bộ kernel của chính nó, và vòng PLD trên CUDA graph chạy nhanh hơn giải mã exact M=1 cùng kernel 1.14–1.32$\times$ với 0 lệch trong 1200 cặp prompt-mô hình. Mức tăng là khiêm tốn, phụ thuộc workload và chỉ có với PLD; mô hình nháp thần kinh không có lợi trên đích 3B. Tham chiếu exact là bộ kernel mới, khác bitsandbytes gốc ở phần lớn chuỗi nhưng không phát hiện giảm độ chính xác tác vụ ở cỡ mẫu nhỏ.

Mã nguồn. `src/zassd/kernels/`, `scripts/wp*`, số liệu ở `results/kernels/`; kế hoạch và bản thảo bài báo ở `docs/PLAN_SHAPE_INVARIANT_KERNELS.md` và `docs/PAPER_DRAFT_KERNELS.m`

Kết luận và hướng phát triển

8.1 Kết luận

Đồ án đã xây dựng và khảo sát ZASSD, một hệ thống giải mã suy đoán không cần nạp checkpoint draft độc lập. Hệ thống gồm đường vanilla, Prompt Lookup Decoding (PLD), self-speculation bằng layer-skip, bộ định tuyến kết hợp các hành động, target verification và quản lý KV cache. Các phép đo được tổ chức theo cohort riêng để phân biệt kết quả hiệu năng, numerical fidelity và chẩn đoán kernel.

Trong benchmark paired 150 prompt trên Qwen2.5-3B-Instruct NF4, PLD đạt speedup end-to-end trung bình paired $1.063\times$ so với vanilla (CI bootstrap 95%: $[1.036, 1.092]$); router đạt $1.050\times$, nhưng chậm hơn PLD theo tỷ số paired 0.989 (CI: $[0.985, 0.993]$). Kết quả cho thấy PLD có thể tăng throughput trong cấu hình đã thử, còn router chưa chứng minh lợi ích so với baseline cố định mạnh nhất. Exact sequence match 83.3% cũng cho thấy speedup không đồng nghĩa với bảo toàn hoàn toàn chuỗi đầu ra.

Các phân tích bổ sung cho thấy chi phí cycle, backend và cách tổ chức verification đều liên quan đến hiệu năng và fidelity. Tuy nhiên, cohort chẩn đoán được chọn có chủ đích, phép profiling chỉ chạy trên một prompt, và verification tuần tự chỉ khôi phục sequence identity trong các mẫu đã kiểm tra. Vì vậy, đóng góp chính của đồ án là một hệ thống có thể so sánh các đường giải mã trong cùng framework cùng một đánh giá thực nghiệm có giới hạn rõ ràng; kết quả chưa bảo đảm khả năng khái quát sang cấu hình khác.

Chương 6 mở rộng phần đánh giá sang tính exact: một vòng giải mã suy đoán B2 trên NF4 tái tạo đúng từng token và KV của greedy bitsandbytes gốc ở mọi prompt đã kiểm, gồm hai cohort held-out GSM8K mỗi cohort 50 prompt dùng đúng một lần (khớp $50/50$ ở cả hai), không dùng thêm bộ nhớ đáng kể (0.07 MiB ở đỉnh) và nhanh hơn bitsandbytes eager khoảng 1.12–$1.15\times$. Mức tăng này khiêm tốn và bị giới hạn bởi chi phí khởi chạy kernel của CPU và acceptance của PLD; B2 chậm hơn một baseline `torch.compile` (khoảng $0.91\times$ tốc độ của nó) vốn không exact, nên đóng góp nằm ở tính exact và quy trình bằng chứng chứ không ở tốc độ tuyệt đối.

Chương 7 đi tiếp từ giới hạn tốc độ của B2: bộ kernel 4-bit bất biến theo số hàng (LM

head, attention, RMSNorm, GEMM NF4) cho phép kiểm tra theo khối mà vẫn exact theo bộ kernel của chính nó. Vòng PLD exact trên CUDA graph cho 0 lệch token và KV trên 1200 cặp prompt-mô hình, tám mô hình và ba tập dữ liệu, nhanh hơn baseline exact $M{=}1$ cùng kernel 1.14–$1.32\times$ và nhanh hơn vòng B2 chạy từng hàng khoảng 1.26–$1.39\times$. Cái giá là tham chiếu exact khác bitsandbytes gốc ở phần lớn chuỗi (chỉ 30–44% giống hệt), dù không phát hiện giảm độ chính xác tác vụ ở cỡ mẫu nhỏ; và mô hình nháp thần kinh không có lợi trên đích 3B ($0.66\times$). Đóng góp nằm ở bộ kernel, định nghĩa exact theo bộ kernel và quy trình bằng chứng, không ở tốc độ tuyệt đối.

8.2 Hạn chế

Kết quả benchmark chính được thu trên một NVIDIA RTX 4050 Laptop GPU, một mô hình Qwen2.5-3B-Instruct, NF4/FP16 compute và SDPA. Số prompt benchmark chính là 150, nhưng các cohort router, numerical fidelity và profiling nhỏ hơn và dùng giao thức khác. Nhiệt độ GPU thay đổi trong các snapshot của benchmark; dù thứ tự phương pháp được xoay vòng, phép đo một thiết bị vẫn có thể chịu ảnh hưởng nhiệt và trạng thái hệ thống.

Một số nhóm numerical fidelity được chọn có chủ đích từ các prompt đã có mismatch trước đó; tỷ lệ đúng/sai ở nhóm này phục vụ chẩn đoán, không ước lượng tỷ lệ lỗi trong traffic tự nhiên. Cohort held-out 14 prompt thuộc cùng prepared-input artifact, không phải đánh giá ngoài độc lập. Phép Nsight có instrumentation và chỉ bao phủ một prompt, còn các thống kê cycle gộp không lưu số candidate draft cho từng cycle. Chưa có đủ bằng chứng để khái quát sang GPU, model, quantization, attention backend hoặc workload khác.

Đối với Chương 6, bằng chứng đều từ một GPU và một bộ công cụ (bitsandbytes 0.50.0, Triton 3.6.0, PyTorch 2.11.0), chủ yếu một họ mô hình (Qwen2) và một backend lượng tử hóa, batch 1, greedy. Tính exact dựa trên bằng chứng hữu hạn chứ chưa phải chứng minh: còn mở sự bằng nhau với kernel bitsandbytes trên mọi đầu vào (mã nguồn CUDA đọc theo nghĩa đen khác với hành vi của bản nhị phân), độc lập của dispatch PyTorch với số hàng, độ ổn định qua phiên bản Triton/ptxas/CUDA và GPU, và kiểm ở mức PTX.

Cuối cùng, exact match với greedy vanilla chỉ đo đồng nhất token ID trong cùng cấu hình. Nó không thay thế đánh giá chất lượng tác vụ, và numerical equivalence của KV/logits cũng không thể được suy ra chỉ từ việc chuỗi đầu ra giống nhau. Các phép đo bộ nhớ dùng peak allocated memory của PyTorch trong một cohort riêng; chúng không đại diện cho toàn bộ VRAM do driver/NVML báo cáo.

8.3 Hướng phát triển

Các công việc tiếp theo được ưu tiên như sau:

1. Mở rộng benchmark paired sang nhiều GPU, model và backend; lặp phiên chạy độc lập để định lượng biến thiên giữa phiên, nhiệt độ và thứ tự chạy.

2. Bổ sung số liệu theo cycle: candidates, accepted/emitted tokens, thời gian không chồng lấn và action fallback. Dùng chúng để kiểm định cost model độc lập.

3. Cải thiện router bằng baseline PLD mạnh đã hiệu chỉnh trên tập development, sau đó đánh giá trên held-out traffic; báo cáo riêng lựa chọn action, overhead, speedup và fidelity.

4. Tiếp tục phân tích sai khác số học giữa batched và sequential verification, KV history, NF4 và các backend khác; thiết kế chế độ fallback có thể bảo toàn sequence identity với overhead được đo rõ.

5. Đo thêm chất lượng tác vụ, latency đầu-cuối, VRAM allocated/reserved/NVML và prompt/đầu ra đa dạng.

6. Chạy bộ kernel bất biến trên GPU khác và mô hình $\geq$ 7B (nơi mô hình nháp thần kinh có thể có lợi), thử bitsandbytes 0.50.1/0.50.2, hỗ trợ $head_dim$ 96 (Phi-3.5), và chứng minh hình thức hoặc kiểm ở mức PTX tính bất biến.

7. Kiểm tra vòng giải mã exact B2 trên GPU và phiên bản Triton/bitsandbytes khác, ở mức PTX, trên backend lượng tử hóa và họ mô hình khác, và hoàn thiện lập luận theo toán tử; mọi lần nâng cấp bitsandbytes cần chạy lại phép mô phỏng thứ tự rút gọn của kernel.

Mục tiêu là xác định được miền cấu hình mà lợi ích throughput của speculative decoding đủ lớn, có thể tái lập và đáp ứng ngưỡng chính xác của ứng dụng, thay vì giả định một phương pháp duy nhất luôn tốt nhất.

Tài liệu tham khảo

[1] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems*, 2018.

[2] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[3] Charlie Chen, Sebastian Borgeaud, et al. Accelerating large language model decoding with speculative sampling. In *arXiv preprint arXiv:2302.01318*, 2023.

[4] Apoorv Saxena. Prompt lookup decoding. *GitHub repository*, 2023.

[5] Zhenyu He, Zexuan Zhong, Tianle Cai, et al. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 1582–1595, 2024.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM Symposium on Operating Systems Principles*, pages 611–626, 2023.

[7] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, et al. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[8] Jun Zhang, Jue Wang, Huan Jiang, et al. Draft and verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2409.18824*, 2024.

[9] Fangcheng Liu, Yehui Tang, Zhen Liu, et al. Kangaroo: Lossless self-speculative decoding via double early exiting. In *Advances in Neural Information Processing Systems*, 2024.

[10] Xin Men, Mingge Xu, Bingning Zhang, et al. Shortgpt: Layers in large language models are more redundant than you expect. In *International Conference on Learning Representations*, 2024.

[11] Longze Chen, Renke Shan, Huiming Wang, et al. Clasp: In-context layer skip for self-speculative decoding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 31608–31618, 2025.

[12] Tianle Cai, Yuhong Li, Zheng Geng, et al. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[13] Yuhui Li, Fangyun Wei, Chao Zhang, et al. Eagle: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[14] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, pages 3519–3529, 2019.

[15] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[16] Raja Gond, Aditya K Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2026.

[17] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab blog, 10 September 2025, 2025.

[18] Jiayi Yuan et al. Understanding and mitigating numerical sources of nondeterminism in LLM inference, 2025.

[19] Chordiya. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware, 2026.

[20] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[21] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.

[22] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.